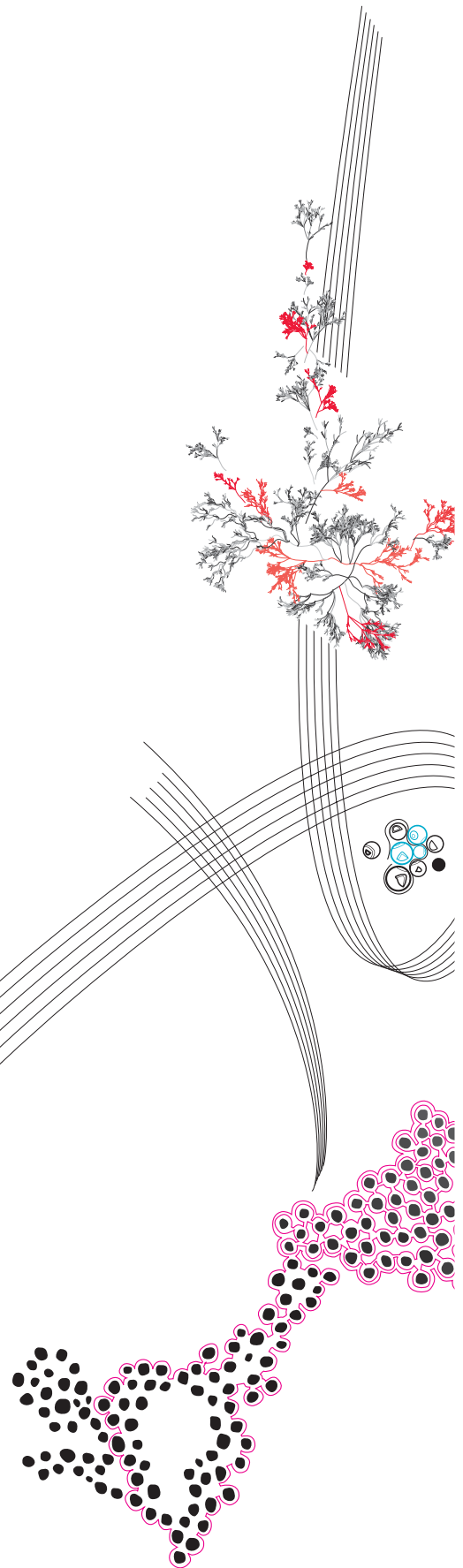




MSc Computer Science
Final Project

A Hierarchical Time-Aware Transformer for Flow-Level Network Intrusion Detection

Xiaoyan Xiong

Supervisor: Le Viet Duc

July, 2026

Department of Computer Science
Faculty of Electrical Engineering,
Mathematics and Computer Science,
University of Twente

Contents

1	Introduction	1
2	Related Work	2
2.1	Signature-Based and Statistical Flow-Based Detection	2
2.2	Deep Learning for Packet and Flow Sequences	3
2.3	Transformer-Based Traffic Representation	3
2.4	Self-Supervised and Encrypted-Traffic Representation Learning	4
2.5	Temporal, Relational, and Cross-Flow Context	4
2.6	Research Gap	6
3	Dataset Construction and Preprocessing	6
3.1	Data Provenance and Scope	6
3.2	PCAP-to-CSV Extraction	7
3.3	Alert-Derived Label Construction	7
3.4	Raw Dataset Composition	8
3.5	Raw Packet and Flow Schemas	9
3.6	Integrity Checks and Packet-Flow Alignment	10
3.7	Feature Selection and Leakage Control	11
3.8	Train-Fitted Feature Transformations	11
3.9	Packet-Sequence Construction	12
3.10	Chronological Train-Validation-Test Partition	13
3.11	Stage 1 Artifacts Supplied to Stage 2	13
3.12	Validity, Privacy, and Reproducibility Considerations	13
4	Proposed Methodology	14
4.1	Architectural Overview	15
4.2	Stage 1: Intra-Flow Time-Aware Representation	16
4.2.1	Packet-Record Projection	16
4.2.2	Cumulative Time-Aware Encoding	16
4.2.3	Flow-Statistics Encoder	17
4.2.4	Bounded Flow-Conditioned Token Modulation	18
4.2.5	Masked Transformer Encoder	18
4.2.6	Hybrid Mask-Aware Pooling	19
4.2.7	Stage 1 Fusion Variants	19
4.3	Stage 2: Causal Inter-Flow Context Modelling	20
4.3.1	Embedding Interface and Chronological Order	20
4.3.2	Relation-Specific Causal Contexts	20
4.3.3	Context-Age Positional Encoding	21
4.3.4	Target-Query Gated Multi-Head Attention	22
4.4	Imbalance-Aware Objective and Stage Interface	22
4.4.1	Focal Objective and Controlled Sampling	22
4.4.2	Sequential Stage Interface	23
5	Experimental Methodology	23
5.1	Evaluation Manifests and Leakage Controls	23
5.2	Experimental Matrix	24
5.2.1	Stage 1 Baselines	24

5.2.2	Stage 1 Ablations and Sensitivity Tests	25
5.2.3	Stage 2 Baselines and Context Ablations	25
5.3	Training, Model Selection, and Thresholding	25
5.3.1	Principal Training Configuration	25
5.3.2	Checkpoint and Decision-Threshold Selection	26
5.4	Evaluation Metrics	26
5.4.1	Class-Wise and Macro-Averaged Measures	27
5.4.2	False-Positive Rate and Operational Error Counts	27
5.4.3	Threshold-Independent Ranking Measures	27
5.5	Efficiency and Resource Measurement	28
6	Results and Discussion	29
6.1	Stage 1: Intra-Flow Representation	29
6.1.1	Architecture and Temporal-Encoding Effects	29
6.1.2	Packet and Flow-Statistics Integration	31
6.1.3	Sequence Length and Packet Selection	32
6.2	Stage 2: Inter-Flow Context	33
6.2.1	Gain over Independent Flow Classification	33
6.2.2	Which Relation Defines Useful Context?	33
6.2.3	How Much Context Is Useful?	34
6.2.4	Stage 2 Architecture Comparison	35
6.2.5	Efficiency and Operational Trade-Offs	36
6.3	Cross-Stage Discussion	36
7	Conclusion and Future Work	37
7.1	Conclusions	37
7.2	Limitations and Scope	37
7.3	Future Work	38
7.3.1	Resource-Efficient Self-Supervised Pre-Training	38
7.3.2	Attack-Adaptive Relation-Aware Context Modelling	38
7.3.3	Streaming Deployment, Efficiency, and Robustness	39

Abstract

Many flow-level network intrusion detection pipelines reduce each connection to aggregate measurements and then classify flows in isolation. This efficient abstraction suppresses packet ordering and non-uniform timing, and it can obscure evidence that becomes visible only across related connections. To retain both levels of structure, this thesis develops a hierarchical detector that does not consume payload content. Aligned packet records and flow statistics are extracted from an enterprise PCAP through a custom Suricata output module. Stage 1 combines positional and cumulative-time information when encoding the ordered packet metadata, conditions the packet tokens on flow statistics, and exports a fused representation for each flow. In Stage 2, the target representation queries a causal history of earlier related flow embeddings before the final prediction is produced.

Chronological partitions are used throughout the evaluation. Because positive flows are scarce, the analysis gives priority to macro-F1, PR-AUC, attack-class performance, confusion matrices, and false-positive rate. On the controlled Stage 1 flow test manifest, the complete fused representation reaches a macro-F1 of 0.8954 and a PR-AUC of 0.8763, exceeding the aggregate-flow, recurrent, convolutional, and encoding controls. On the separate Stage 2 flow manifest, causal source-host context raises macro-F1 from 0.8954 to 0.9275 and PR-AUC from 0.8763 to 0.9345 while reducing both types of classification error. The target-query model also surpasses the no-context, vanilla Transformer, LSTM, GRU, and CNN-LSTM alternatives, with model-level latency comparable to that of the vanilla Transformer. Taken together, the results indicate that explicit intra-flow timing and causal inter-flow context supply complementary evidence. These findings concern in-domain performance on a single alert-labelled organisational capture; variation across training runs and external generalisation have yet to be established.

Keywords: Network intrusion detection, packet sequence, time-aware Transformer, flow representation, cross-flow context, class imbalance

1 Introduction

Enterprise systems, cloud platforms, mobile networks, and Internet-of-Things (IoT) deployments collectively produce large and heterogeneous traffic streams. Continuous monitoring is needed to identify activities such as reconnaissance, exploitation attempts, botnet communication, distributed denial-of-service (DDoS), and multi-stage intrusion campaigns. Network Intrusion Detection Systems (NIDSs) consequently form an important layer of cyber-security infrastructure [1, 2]. Their practical design nevertheless remains difficult. Contemporary attacks may be distributed, adaptive, and temporally extended, while encryption restricts direct payload inspection and increases reliance on headers, metadata, and behavioural traffic characteristics [3, 4].

Operational deployments continue to rely on signature engines such as Snort and Suricata because their rules are interpretable, computationally efficient, and effective against known threats [1, 5]. Suspicious traffic is recognised through patterns designed and maintained by domain experts. Such rules remain valuable, but their coverage is necessarily bounded: unseen attacks and evolving variants may not match an existing signature, and malicious intent may emerge only from temporal or contextual regularities spanning several flows [2, 6]. Rule-based analysis alone is therefore unlikely to provide sufficient coverage as traffic volumes and attack behaviours continue to diversify.

Learning-based NIDSs seek greater adaptability by estimating decision boundaries from observed traffic rather than depending exclusively on predefined rules. In conventional pipelines, a bidirectional flow is represented by aggregate features such as its duration, packet and byte counts, rates, and inter-arrival-time statistics [7, 8]. Compact fixed-length vectors support efficient large-scale processing, but the compression removes detail about the process that generated each flow. Packet order, bursts, directional transitions, and uneven temporal gaps can all disappear during aggregation. For attacks whose distinguishing evidence lies in how a connection unfolds rather than only in its total volume, that information loss is consequential.

Richer traffic representations have been investigated with convolutional and recurrent neural networks, LSTMs, GRUs, and autoencoder-based architectures [9, 10, 11]. These models can capture non-linear feature interactions while reducing manual feature engineering. Yet their inputs often remain aggregate flow records. In some sequence-based studies, the sequence is formed from fixed-size flow attributes rather than the packets observed within a connection. Such formulations preserve only part of the natural hierarchy of network traffic: packets constitute flows, and related flows in turn express host- or endpoint-level behaviour.

Self-attention offers a suitable mechanism for addressing this hierarchy because distant sequence elements can interact directly and training can be parallelised [12]. Transformer-based systems have consequently achieved competitive results in traffic classification and intrusion detection [13, 14, 15]. Time-aware variants show, in particular, that detection can benefit when order and elapsed time are encoded as distinct signals [14]; FlowTransformer demonstrates the architectural flexibility available for flow-based NIDS evaluation [15]. Even so, a substantial portion of this literature continues to represent each flow as one flat item. Packet-level dynamics inside a flow and contextual dependencies between flows are seldom modelled together.

To address this gap, the thesis organises detection into two stages. Raw PCAP traffic is first converted into aligned packet records and flow statistics by a custom Suricata output module. Stage 1 derives a flow representation from packet order, elapsed time, and aggregate measurements. A bounded causal history of related representations is then used

by Stage 2 to refine the flow-level decision. Separating these operations permits intra-flow representation learning and inter-flow reasoning to be assessed independently without changing the final prediction granularity.

The central research problem of this thesis is therefore:

How can network intrusion detection benefit from packet-level temporal modelling and cross-flow contextual modelling while preserving the hierarchical structure of network traffic?

To answer this question, the thesis studies two research questions:

- **RQ1:** Does time-aware packet-sequence modelling produce more informative flow representations than flow-level aggregation alone?
- **RQ2:** Does incorporating cross-flow context improve flow-level intrusion detection compared with independent per-flow classification?

The thesis makes three principal contributions:

- An aligned PCAP-to-learning pipeline built around a custom Suricata output module, with explicit separation among predictive variables, context identifiers, and alert-derived targets.
- A Stage 1 representation that integrates packet position, cumulative elapsed time, mask-aware self-attention, and bounded flow-conditioned token modulation.
- A causal Stage 2 target-query attention model that draws evidence from relation-specific histories while excluding future flows and historical labels.

Given the low prevalence of the positive class, macro-F1, PR-AUC, attack-class performance, and false-positive rate carry more interpretive weight than accuracy alone.

2 Related Work

Research on network intrusion detection spans manually specified signatures and data-driven models learned from packet traces, flow records, or ordered network events. The literature can be organised along two dimensions: representation granularity and contextual scope. Inputs range from packet bytes and header sequences to aggregate flow statistics and learned embeddings. Most systems assign labels independently to individual packets or flows; comparatively fewer studies represent temporal or relational dependencies across several flows.

2.1 Signature-Based and Statistical Flow-Based Detection

Expert-authored rules are the basis of signature-driven NIDSs. Snort introduced a lightweight, extensible design for real-time packet inspection [1]; Suricata extends this operational model with multi-threaded processing and programmable logging [5]. Their alerts are interpretable and rule behaviour can be controlled, which preserves their operational relevance. Coverage, however, depends on maintaining an appropriate signature set, and new

or behaviourally altered attacks may fall outside it. Sommer and Paxson place this limitation in a broader methodological context: learning-based NIDS research must account for both the open-world nature of traffic and the semantic distance between laboratory benchmarks and deployed networks [2].

Classical machine-learning pipelines usually group packets into bidirectional flows and describe each flow through duration, volume, rate, TCP-flag, and inter-arrival summaries. Ahmed et al. and Buczak and Guven review the use of decision trees, support-vector machines, nearest-neighbour methods, clustering, and ensembles for anomaly and intrusion detection [8, 7]; conventional models can also benefit from feature selection and hybrid classification [16]. Aggregation converts a variable-length conversation into an efficient fixed-dimensional vector. That convenience entails a loss of process information: similar totals may arise from different packet orders, directional changes, burst structures, or temporal gaps. A model restricted to aggregate statistics cannot observe those generating dynamics directly.

2.2 Deep Learning for Packet and Flow Sequences

Deep architectures learn non-linear traffic representations and thereby reduce reliance on manually selected feature combinations. Recurrent NIDSs maintain a hidden state over ordered observations; Yin et al., for example, demonstrated that recurrent neural networks can capture sequential dependencies in intrusion data [9]. Shone et al. paired a non-symmetric deep autoencoder with a supervised classifier to derive compact representations [10]. Kitsune pursued lightweight online anomaly detection through an ensemble of incrementally processed autoencoders [11]. Local pattern extraction and recurrent temporal modelling have also been combined in CNN-LSTM hybrids [17].

Architecture alone does not determine whether temporal structure is learned; the construction of the sequence is equally important. An LSTM or GRU is sometimes applied across the dimensions of one flow vector, so the resulting “sequence” represents feature order rather than chronological packets or flows. Feature interactions may be learned in this setting, but network time is not necessarily represented. NetSentry adopts a more direct temporal formulation by processing ordered observations for early detection of large-scale attacks [18]. Its findings support the value of temporal correlation beyond independent record classification. Recurrent computation nevertheless proceeds sequentially, and compressing a long or irregularly sampled history into one final state can create an information bottleneck.

2.3 Transformer-Based Traffic Representation

By replacing recurrent transitions with self-attention, Transformers permit direct interaction between distant tokens and support parallel training [12]. Both properties are relevant when evidence is separated by numerous packets or flows. RTIDS applied this mechanism to robust intrusion classification on flow-oriented benchmarks [13]. An encoder-based Transformer was later examined for cloud network intrusion detection by Long et al. [19], whereas IDS-INT combined Transformer transfer learning, imbalance handling, and downstream neural components [17].

GTID combines packet- and session-level features in a time-aware Transformer whose temporal coordinate is derived from inter-packet intervals rather than ordinal position alone [14]. FlowTransformer takes a complementary, modular approach, enabling input encodings, Transformer blocks, classification heads, parameter counts, and runtime to be

examined on flow-based NIDS datasets [15]. The memory flow Transformer extends attention to longer flow histories and complex relations in large sequential inputs [20]. Together, these studies show that dependencies lost in a single aggregate vector can be recovered by attention-based models. They also make clear that encoding, pooling, sequence definition, and computational cost are at least as consequential as the choice of model family.

Three unresolved distinctions motivate the present hierarchy. Ordinal adjacency does not reveal whether two packets are separated by microseconds or seconds. Packet sequences and aggregate statistics also describe different properties of the same connection. Finally, even a rich representation of one flow cannot establish on its own whether that flow belongs to a distributed host-level pattern. The proposed design treats these as separate representational requirements.

2.4 Self-Supervised and Encrypted-Traffic Representation Learning

As encryption restricts payload inspection, traffic analysis has increasingly turned to packet sizes, directions, timing, header metadata, and learned latent representations. Surveys of encrypted-traffic analysis document this change and the expanding role of deep pre-training [3, 4]. ET-BERT transfers bidirectional Transformer pre-training to contextualised datagram representations for encrypted-traffic classification [21]. Masked autoencoders learn from unlabelled records by reconstructing concealed parts of an input [22, 23], while contrastive objectives improve label efficiency by aligning compatible traffic views and separating incompatible ones [24].

Recent work has also widened the range of admissible representations. CoTNeT applies contextual Transformer processing to encrypted-traffic features [25]; DoLLM reformulates flow records for large-language-model-based DDoS detection [26]. Both illustrate that plaintext payloads are not a prerequisite for useful representation learning. Nevertheless, encrypted-traffic classification should not be conflated with intrusion detection. An application’s identity can remain stable even when malicious behaviour is sparse, evolving, or distributed over several connections.

2.5 Temporal, Relational, and Cross-Flow Context

Conditional independence between flows is a convenient assumption for batching and online inference, but many attack processes violate it. Scanning, password guessing, command and control, DDoS, and lateral movement may each generate a collection of individually ambiguous flows whose shared source, destination, endpoint, or temporal pattern is informative. NetSentry demonstrates the value of preserving observation order [18]. Graph-based NIDSs instead encode host and flow relations explicitly: Deng et al. use graph convolution over flow topology for label-limited IoT detection [27], and Xu et al. combine graph neural networks with self-supervised learning for network-flow intrusion detection [28]. Zhong et al. survey this expanding area and emphasise the capacity of graph models to represent structural dependence [29].

Relational inductive bias is expressive, although maintaining a large communication graph can increase memory demand, latency, and deployment complexity. Window-based sequences occupy a different point in this design space. One recent Transformer groups IoT flows into temporal windows and represents every flow as a token, allowing inter-flow dependence to be learned through self-attention [30]. A review of contextualised NetFlow

NIDSs further distinguishes temporal, relational, multimodal, and multi-resolution context, and stresses that causal ordering and split isolation must be preserved during evaluation [31].

The modelling scope of representative Transformer-based approaches is contrasted in Table 2.1. Numerical ranking would be inappropriate because the underlying datasets, labels, preprocessing procedures, and evaluation protocols differ. In this comparison, *explicit elapsed time* requires direct encoding of timestamps or inter-arrival intervals; ordinal position or temporal-window membership alone is not sufficient.

TABLE 2.1: Qualitative scope of representative Transformer-based traffic models.

(a) Representation and temporal modelling			
Method	Primary input	Packet order	Elapsed time
FlowTransformer [15]	Flow records and statistics	No	Not explicitly encoded
GTID [14]	Packet headers and payload n-grams within a session	Yes	Inter-packet intervals
ET-BERT [21]	Datagram bytes grouped into BURSTs	Byte and BURST order	Order only; no packet intervals
Flow-to-window Transformer [30]	Flow records in fixed-duration windows	No	Window order; no packet intervals
Proposed	Packet-header sequence and flow statistics	Yes	Packet intervals

(b) Context, payload use, and task			
Method	Inter-flow context	Payload use	Primary task
FlowTransformer [15]	Ordered flow sequence	None	Flow-level NIDS
GTID [14]	None	Payload n-grams	NIDS
ET-BERT [21]	None	Encrypted bytes	Encrypted-traffic classification
Flow-to-window Transformer [30]	Temporal window	None	IoT NIDS and traffic classification
Proposed	Time, source, destination, or endpoint	No payload content	Flow-level NIDS

Notes: Entries describe the principal configuration reported in each study; extensible frameworks may support other variants. “No payload content” excludes direct payload bytes but not scalar metadata such as payload length.

This qualitative contrast identifies a gap without treating results from incompatible experimental settings as directly rankable. FlowTransformer and the flow-to-window model capture dependencies among flow records but do not reconstruct the packet dynamics within a flow. GTID represents both packet order and inter-packet time, although it processes one session at a time and incorporates payload-derived n-grams. ET-BERT learns byte- and BURST-level representations from encrypted raw traffic, but is primarily designed for encrypted-traffic classification rather than cross-flow intrusion detection. Within the selected comparison, only the proposed framework combines payload-content-independent packet modelling, explicit elapsed time, flow-statistical conditioning, and con-

figurable context across flows. These complementary requirements motivate the separation of Stage 1 from Stage 2.

2.6 Research Gap

Aggregate flow statistics, sequential encoders, time-aware attention, self-supervised traffic representations, and graph- or window-based context have each proved useful. They are commonly investigated in isolation, however. Statistical flow models trade packet dynamics for efficiency; packet-sequence encoders generally stop at independent flow classification; and cross-flow models often begin with raw flow vectors rather than representations learned from timing-aware packet sequences.

The proposed hierarchy assigns these two scales of evidence to different stages. Packet order and elapsed time are represented within Stage 1 and conditioned by flow statistics. Stage 2 operates on the resulting embeddings under explicit temporal and endpoint-based context policies. In this way, *intra-flow* packet dynamics are connected to *inter-flow* attack context without collapsing them into one input representation. Separate ablations then test the contribution of time-aware packet modelling and the additional value of contextual flow history.

3 Dataset Construction and Preprocessing

Two synchronised views of the traffic are required: an ordered set of packet records for every flow and a single statistical record describing that flow. The raw CSV schema is kept distinct from the Stage 1 predictor set and from identifiers used only to construct Stage 2 context, since linkage metadata need not be suitable predictive variables. Alignment and sequence assembly are deterministic, whereas every transformation estimated from data is fitted on the training partition. This organisation supports reproducibility and limits information leakage during NIDS evaluation [32, 33, 34, 35].

3.1 Data Provenance and Scope

Experiments are conducted on `ar002_et12_20260511_001.pcap`, an approximately 8.62 GB packet capture supplied by the company. Organisational staff acquired the trace from the company network with an internal traffic-collection tool; no laboratory traffic was generated for this thesis. CICIDS-style datasets schedule benign profiles and attacks under controlled conditions [36]. By contrast, the present trace records enterprise traffic as it was visible at the collection point. This improves environmental realism but also binds the observed traffic mix, host population, sensor visibility, and attack coverage to one organisation and capture period.

Because the dataset is proprietary, it is not presented as a public benchmark. Sensitive network identifiers, notably IP addresses and ports, remain in the source PCAP and CSV files; only aggregate quantities are reported in the thesis. Before any artefact leaves the authorised environment, host identifiers should be replaced consistently, superfluous endpoint fields removed, and the result reviewed under company data-governance policy. Stage 2 relies on equality relations between hosts, so stable pseudonyms are preferable to removing every identifier, although the pseudonym map itself need not be disclosed. Raw addresses are still retained by the current extraction code in local metadata. Pseudonymi-

sation is therefore a requirement for future release, not a property claimed for the current CSV files.

3.2 PCAP-to-CSV Extraction

Suricata, an open-source network threat detection engine, is used to replay the PCAP [1, 5]. The custom C module at `suricata-plugin/custom-out.c` registers separate packet and flow loggers through Suricata’s low-level output interfaces. Both CSV views are emitted during the same pass. Consequently, packet-to-flow membership does not need to be reconstructed by another tool, and the packet and flow tables retain a common bidirectional `flow_id`.

The construction path is shown in Figure 3.1. Packet rows retain microsecond timestamps and their order within each flow. In parallel, online aggregates are maintained by the flow logger and written once a flow completes. Broad feature families follow CICFlowMeter/CICIDS-style data, with additional indicators recording whether initial TCP-window measurements are available in the forward and backward directions [36].

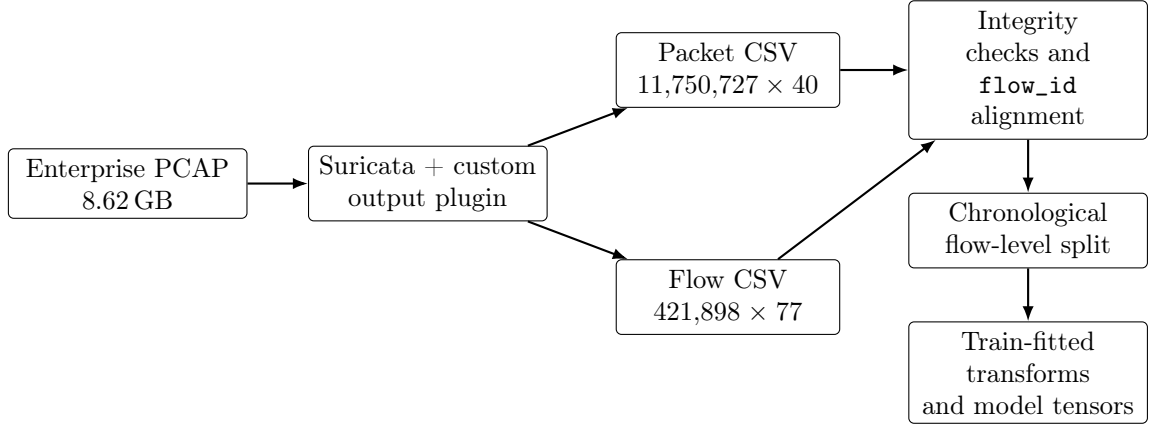


FIGURE 3.1: Construction and preprocessing path from the proprietary PCAP to Stage 1 tensors and Stage 2 metadata.

Neither learning stage receives raw application payload bytes. Although `payload_len` is present, it records only a scalar length derived from packet metadata and contains no application content. The detector is therefore independent of payload content, but not of every payload-related measurement. Suricata signatures, rule identifiers, alert text, and alert counts are likewise excluded from the predictor set.

3.3 Alert-Derived Label Construction

Binary targets are derived by the custom logger from Suricata’s alert state. If $a(p_i)$ is the number of alerts associated with packet p_i , its packet-level target is

$$y_i^{\text{pkt}} = \mathbb{I}[a(p_i) > 0], \quad (3.1)$$

where $\mathbb{I}[\cdot]$ denotes the indicator function. A positive flow contains at least one packet that triggered an alert:

$$y_f = \max_{p_i \in \mathcal{P}_f} y_i^{\text{pkt}}, \quad (3.2)$$

Here, $\mathcal{P}_f$ contains all packets assigned to flow f . Accordingly, `packet_label=1` identifies an alert-triggering packet, whereas `label=1` marks an alert-associated flow.

These programmatically assigned targets are *weak labels*, not ground truth verified independently by experts [37]. Label 0 signifies only that the configured Suricata rules emitted no alert; benignity is not thereby established. Positive targets similarly inherit false alarms from the deployed ruleset. The reported experiments thus evaluate the learning and contextualisation of behaviour associated with Suricata alerts, not guaranteed coverage of every intrusion or zero-day attack. Errors in the labelling process may influence both model fitting and test estimates and must be considered when the findings are interpreted [38].

Direct target leakage is prevented by excluding `packet_label`, `label`, and all Suricata alert metadata from model inputs. Exact replication of the weak-labelling function would additionally depend on the Suricata version, a ruleset snapshot, the custom-module revision, its configuration, and the PCAP checksum. Where this provenance was not retained, the omission is reported in Chapter 5 rather than filled retrospectively.

3.4 Raw Dataset Composition

Table 3.1 reports the dimensions of the aligned outputs. There are 11,750,727 packet rows across 40 columns and 421,898 flow rows across 77 columns. Before truncation, their count ratio is approximately 27.85 logged packets per flow. This mean should not be read as evidence of uniform flow length: packet counts are typically heavy-tailed, making an explicit upper bound on sequence length necessary.

TABLE 3.1: Raw extracted dataset summary.

Data view	Rows	Columns	Primary role
Packet CSV	11,750,727	40	Ordered packet-header, protocol, direction, and inter-arrival observations within flows
Flow CSV	421,898	77	One flow-level target, aggregate statistics, and context-construction metadata per flow

Among the 421,898 flow records, 405,254 carry label 0 and 16,644 carry label 1, as detailed in Table 3.2. Hence, the positive prevalence and class-imbalance ratio are

$$\pi_1 = \frac{16,644}{421,898} = 0.03945 \quad (3.945\%), \quad (3.3)$$

$$\text{IR} = \frac{405,254}{16,644} = 24.35 : 1. \quad (3.4)$$

TABLE 3.2: Flow-level alert-derived label distribution.

Label	Interpretation	Count	Proportion
0	No Suricata alert observed in the flow	405,254	96.055%
1	At least one alert-triggering packet	16,644	3.945%
Total		421,898	100.000%

At this prevalence, predicting label 0 for every flow would yield approximately 96.055% accuracy and zero positive recall. Accuracy and frequency-weighted averages are therefore treated as secondary. Macro-precision, macro-recall, macro-F1, ROC-AUC, PR-AUC, confusion-matrix counts, and false-positive rate provide the main evidence. PR-AUC is especially informative for a rare positive class [39]. Operationally, false positives also warrant direct attention because even a small fraction of 405,254 negative flows can translate into a large number of alerts.

3.5 Raw Packet and Flow Schemas

All raw columns are grouped by function in Tables 3.3 and 3.4. These groups describe the extracted schema and should not be interpreted as the final predictor set.

TABLE 3.3: Complete packet CSV schema grouped by function.

Group	Raw columns
Record identity and time	record_type, packet_id, timestamp_us, ts_sec, ts_usec, flow_id
Endpoints and protocol	direction, ip_version, src_ip, dst_ip, src_port, dst_port, protocol, l7_proto
Length and header metadata	pkt_len, ip_len, payload_len, ttl_or_hop_limit, tcp_window, tcp_header_len, udp_len
Transport indicators	has_tcp, has_udp, has_icmp
TCP state and flags	tcp_seq, tcp_ack, tcp_flags, tcp_fin, tcp_syn, tcp_rst, tcp_psh, tcp_ack_flag, tcp_urg, tcp_ece, tcp_cwr
ICMP metadata	icmp_type, icmp_code
Inter-arrival time	flow_iat_us, direction_iat_us
Target	packet_label

The two packet-level timing fields are not interchangeable. `flow_iat_us` measures elapsed time since the preceding packet in the bidirectional flow; `direction_iat_us` refers instead to the preceding packet travelling in the same direction. Values are expressed in microseconds and set to zero when no applicable predecessor exists.

TABLE 3.4: Complete flow CSV schema grouped by function.

Group	Raw columns
Identity and time	flow_id, flow_start_timestamp_us, flow_end_timestamp_us
Endpoints and protocol	source_ip, source_port, destination_ip, destination_port, protocol
Directional volume	total_fwd_packets, total_backward_packets, total_length_of_fwd_packets, total_length_of_bwd_packets
Directional length statistics	fwd_packet_length_max, fwd_packet_length_min, fwd_packet_length_mean, fwd_packet_length_std, bwd_packet_length_max, bwd_packet_length_min, bwd_packet_length_mean, bwd_packet_length_std

Continued on next page

Table 3.4 – continued from previous page

Group	Raw columns
Overall rate and timing	flow_duration, flow_bytes_per_s, flow_packets_per_s, flow_iat_mean, flow_iat_std, flow_iat_max, flow_iat_min
Directional inter-arrival time	fwd_iat_total, fwd_iat_mean, fwd_iat_std, fwd_iat_max, fwd_iat_min, bwd_iat_total, bwd_iat_mean, bwd_iat_std, bwd_iat_max, bwd_iat_min
Directional flags, headers, and rates	fwd_psh_flags, bwd_psh_flags, fwd_urg_flags, bwd_urg_flags, fwd_header_length, bwd_header_length, fwd_packets_per_s, bwd_packets_per_s
Overall packet-length distribution	min_packet_length, max_packet_length, packet_length_mean, packet_length_std, packet_length_variance
TCP flag counts	fin_flag_count, syn_flag_count, rst_flag_count, psh_flag_count, ack_flag_count, urg_flag_count, cwe_flag_count, ece_flag_count
Ratios, segments, and initial windows	down_up_ratio, average_packet_size, avg_fwd_segment_size, avg_bwd_segment_size, has_init_win_bytes_forward, init_win_bytes_forward, has_init_win_bytes_backward, init_win_bytes_backward, act_data_pkt_fwd, min_seg_size_forward
Active and idle periods	active_mean, active_std, active_max, active_min, idle_mean, idle_std, idle_max, idle_min
Target	label

TCP-specific fields are zero-filled for non-TCP packets, with `has_tcp` distinguishing an inapplicable field from a genuine observed zero. UDP and ICMP presence indicators serve the same purpose. At flow level, `has_init_win_bytes_forward` and `has_init_win_bytes_backward` mark the availability of their corresponding initial-window measurements; a missing measurement is stored as zero.

3.6 Integrity Checks and Packet–Flow Alignment

Preprocessing begins by normalising column names and discarding `record_type`, a logging descriptor with no predictive role. Infinite values are converted to missing entries. Rows are rejected when `flow_id` is zero or missing; valid flow rows also require `flow_start_timestamp_us`, while packet rows require `timestamp_us`, in each case with a nonzero, nonmissing value.

Only one effective flow row is retained for each `flow_id`. When duplicates occur, a stable sort by identifier and start time is applied, the first record is kept, and the number of discarded duplicates is logged. The identifier sets of the packet and flow tables are then intersected so that unmatched rows can be removed from both views. A final sort places packets in `flow_id` and `timestamp_us` order. Together, these operations establish the one-to-many relation

$$f \longleftrightarrow \{p_{f,1}, p_{f,2}, \dots, p_{f,n_f}\}, \quad t_{f,1} \leq t_{f,2} \leq \dots \leq t_{f,n_f}, \quad (3.5)$$

before splitting or feature transformation. Table 3.1 describes the CSV files as extracted, whereas post-cleaning counts for a particular run are determined from its saved split report.

3.7 Feature Selection and Leakage Control

Raw columns are not passed wholesale to the network; the input schema is controlled by configuration. Before one-hot expansion, the principal Stage 1 setup uses the 23 packet predictors listed in Table 3.5. Numerical variables encode packet and header dimensions, categorical variables describe protocol state, and binary indicators retain direction, transport presence, and individual TCP flags.

TABLE 3.5: Packet predictors used by the principal Stage 1 configuration.

Type	Selected raw columns
Numerical (7)	<code>pkt_len</code> , <code>ip_len</code> , <code>payload_len</code> , <code>ttl_or_hop_limit</code> , <code>tcp_window</code> , <code>tcp_header_len</code> , <code>udp_len</code>
Categorical (4)	<code>ip_version</code> , <code>protocol</code> , <code>icmp_type</code> , <code>icmp_code</code>
Binary (12)	<code>direction</code> , <code>has_tcp</code> , <code>has_udp</code> , <code>has_icmp</code> , <code>tcp_fin</code> , <code>tcp_syn</code> , <code>tcp_rst</code> , <code>tcp_psh</code> , <code>tcp_ack_flag</code> , <code>tcp_urg</code> , <code>tcp_ece</code> , <code>tcp_cwr</code>

Stage 1 excludes raw IP addresses and ports, packet and flow identifiers, absolute timestamps, TCP sequence and acknowledgement numbers, the combined TCP-flag integer, `17_proto`, and both targets. Their removal discourages memorisation of site-specific endpoints and prevents identifiers or capture position from becoming target shortcuts. Relative timing is retained separately through a tensor derived from `flow_iat_us`; the absolute packet timestamp is not supplied as an ordinary feature.

When statistical fusion is enabled, a configured subset of Table 3.4 is processed through a separate branch. The principal subset covers duration, directional packet and byte totals, length distributions, rates, inter-arrival summaries, flag counts, header lengths, segment and window measurements, and selected active-period statistics. Availability of the two initial-window measurements is represented by binary indicators. Endpoint identities and ports, absolute flow times, and `label` are omitted. Since the packet-only ablation disables this branch, presence in the raw schema does not imply use in every experiment.

Three metadata fields accompany every Stage 1 embedding: `flow_start_timestamp_us`, `source_ip`, and `destination_ip`. They define relations for Stage 2 and are not Stage 1 predictors. Host equality determines source-host, destination-host, and endpoint contexts, while start time establishes ordering. Addresses are never one-hot encoded as semantic features, and historical labels are not retrieved into context. The required relations are thereby preserved without directly training on host identity.

3.8 Train-Fitted Feature Transformations

Every data-dependent transformation is estimated from the training partition and subsequently applied unchanged to validation and test records. Estimating clipping limits, scaling statistics, or category vocabularies from the complete dataset would expose information from later partitions [40]. Packet and flow variables follow pipelines of the same form, but their fitted parameters remain separate.

Numerical features. After conversion to numeric form, non-finite entries are marked as missing. Selected nonnegative variables with heavy-tailed distributions undergo the transform

$$\tilde{x} = \log(1 + \max(x, 0)). \quad (3.6)$$

Configured variables include packet and payload lengths, TCP window, header length, UDP length, and numerous flow-level counts, durations, byte totals, rates, and inter-arrival summaries. Zero imputation is applied after the logarithmic transform.

For numerical variable j , lower and upper clipping limits are the training quantiles $q_{0.001,j}^{\text{tr}}$ and $q_{0.999,j}^{\text{tr}}$. Winsorisation in every partition is defined by

$$x_{ij}^{\text{clip}} = \min(\max(\tilde{x}_{ij}, q_{0.001,j}^{\text{tr}}), q_{0.999,j}^{\text{tr}}). \quad (3.7)$$

Robust scaling is then applied in the principal configuration:

$$x_{ij}^{\text{scaled}} = \frac{x_{ij}^{\text{clip}} - \text{median}_{\text{tr}}(x_j)}{\text{IQR}_{\text{tr}}(x_j)}, \quad (3.8)$$

where the denominator is computed from the training interquartile range. Logarithmic compression, quantile clipping, and robust scaling limit the influence of extreme bursts without drawing statistics from the test partition.

Categorical features. An explicit MISSING token represents absent categorical values. The one-hot vocabulary is learned from training categories; categories encountered only during validation or testing are ignored, leaving the input dimension unchanged.

Binary features. Binary fields are first converted to numbers. Missing and non-finite entries become zero, as do all non-positive values, while positive entries become one. No one-hot encoding, logarithmic transform, clipping, or scaling is applied to these indicators.

3.9 Packet-Sequence Construction

Packets in aligned flow f are sorted by `timestamp_us`. Under the principal `head` policy, at most $L = 128$ packets are retained. Flows longer than L contribute their first 128 observations; shorter sequences are padded with zeros on the right. Valid positions are recorded in a Boolean mask so that padding is excluded from attention and pooling. The resulting tensor shapes are

$$\mathbf{X} \in \mathbb{R}^{N \times L \times d_p}, \quad \mathbf{T} \in \mathbb{R}^{N \times L}, \quad \mathbf{M} \in \{0, 1\}^{N \times L}, \quad (3.9)$$

with d_p denoting the transformed packet-feature dimension after one-hot encoding.

Nonnegative within-flow inter-arrival times define the timing tensor:

$$T_{f,i} = \log(1 + \max(\Delta t_{f,i}, 0)), \quad \Delta t_{f,i} = \text{flow_iat_us}(p_{f,i}). \quad (3.10)$$

This tensor enters the time-aware encoder separately; absolute timestamps are not concatenated to the packet features.

Two additional policies support the truncation ablation. `head_tail` combines the first $\lfloor L/2 \rfloor$ packets with the final $L - \lfloor L/2 \rfloor$ packets. `random` samples L positions without replacement from a deterministic seed derived from `flow_id` and the global seed, after which the selected packets are returned to chronological order. Separate experiments consider $L \in \{8, 16, 32, 64, 128, 256\}$. Only the retained evidence and associated computation change; flow membership in each split remains fixed.

3.10 Chronological Train–Validation–Test Partition

Unique flows are stably ordered by `flow_start_timestamp_us` before a flow-level split is made. The earliest 70% form the training partition, the following 10% form validation, and the final 20% are reserved for testing. With `stratify=false`, these chronological boundaries are not adjusted to equalise class proportions. For the 421,898 aligned records, this procedure yields the partition sizes in Table 3.6.

TABLE 3.6: Principal chronological flow partition.

Partition	Ratio	Number of flows	Temporal role
Training	70%	295,329	Earliest flows
Validation	10%	42,189	Intermediate flows
Test	20%	84,380	Latest flows
Total	100%	421,898	

Every packet follows the partition assignment of its parent `flow_id`; no flow can span two partitions. Only training-flow packets and statistics are used to fit preprocessing. Validation data support hyperparameter choice, early stopping, and decision-threshold selection, while final evaluation is confined to the test partition. Relative to random row splitting, chronological separation more closely reflects deployment on later traffic and reduces leakage from distributing related packets or temporally adjacent behaviour across partitions [2, 40].

3.11 Stage 1 Artifacts Supplied to Stage 2

Once Stage 1 has been trained, exactly one embedding and one metadata record are exported for each flow in every partition. Metadata comprise `flow_id`, the target, partition name, `flow_start_timestamp_us`, and source and destination identifiers. The embedding provides Stage 2 content; time and host identity are used only to determine which earlier flows are eligible. Retaining the partition name guards against unintended context construction across split boundaries. Targets remain available for training and evaluation but never become context features.

Each Stage 1 run also records the selected features, encoder categories, clipping limits, scaling parameters, sequence policy, random seed, ordered flow identifiers, partition-level class counts, model configuration, and checkpoint identity. These records link every Stage 2 result to the Stage 1 checkpoint and split manifest from which its embeddings originated.

3.12 Validity, Privacy, and Reproducibility Considerations

Although the resulting dataset supports evaluation on a large, temporally ordered enterprise trace, four constraints delimit the conclusions.

L1: Alert-derived ground truth. Label coverage and error reflect a single Suricata rule-set. A negative record is therefore more accurately interpreted as “no alert observed” than as independently verified benign traffic.

L2: Single-trace scope. Evidence from one company capture cannot establish generalisation across organisations, sensors, or extended periods. Evaluation on external PCAPs remains necessary.

L3: Capture visibility. Only traffic observable at the collection point appears in the PCAP. Packet loss, asymmetric routing, encryption, and incomplete flows may change the extracted measurements. Where organisational policy allows, documentation should include the collection tool, interface location, capture interval, packet-loss report, and snap length.

L4: Sensitive identifiers. Stage 2 relations are currently formed from addresses retained in local metadata. Those addresses must be excluded from the thesis, shared CSVs, screenshots, and public metadata. Any released artefact requires stable pseudonymisation verified to preserve equality-based context membership.

Within these limits, the study evaluates a traceable learning pipeline on a large proprietary capture with weak labels. Suricata outputs are not treated as perfect attack truth, the presence of every zero-day behaviour is not assumed, and no claim is made that performance transfers unchanged to other enterprise networks. Tensors, masks, flow measurements, and causal-context metadata together define the interface passed between the two model stages.

4 Proposed Methodology

Two complementary scales of traffic structure are represented by the detector. Stage 1 operates within a bidirectional flow, combining its ordered packets with statistical properties. Stage 2 relates the target flow to eligible historical flows. Sequential training and a fixed-dimensional Stage 1 interface keep intra-flow representation learning distinct from contextual reasoning across flows.

Although the architecture is derived from the Transformer [12], self-attention alone does not define the method. Three mechanisms are combined: (i) a cumulative-time coordinate that represents irregular packet arrival, (ii) an independently encoded statistical branch that modulates packet tokens within a bounded range, and (iii) causal target-query attention from the current flow to historical flows only. The distinction between packet order and elapsed time follows evidence from time-aware NIDS research [14]. Separating within-flow evidence from cross-flow behaviour also addresses attacks whose signal is dispersed over several connections and may be missed when flows are classified independently [18, 15, 30].

The high-level data path appears in Figure 4.1. A time-aware intra-flow Transformer converts packet records into one embedding for each flow. A sequence of these embeddings is then contextualised in Stage 2 to produce a binary decision for the target flow. Figure 4.2 gives a more detailed view of statistical conditioning and target-query attention.

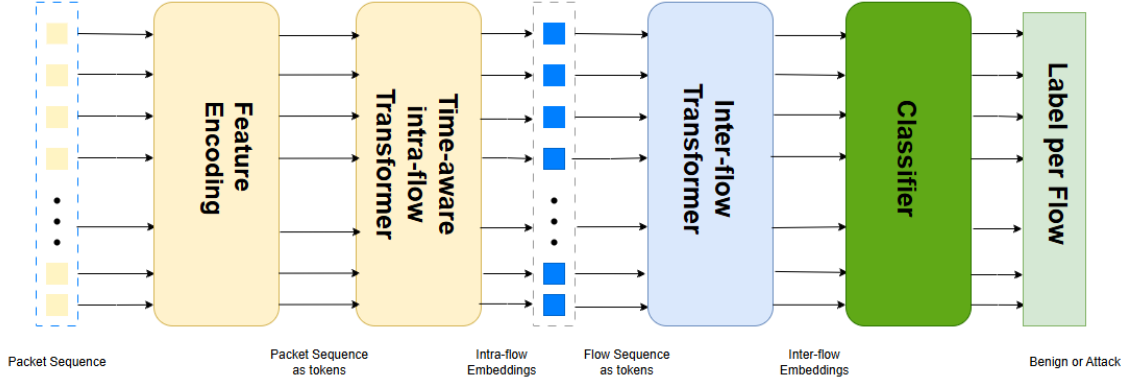


FIGURE 4.1: High-level architecture of the hierarchical detector. Stage 1 transforms a packet sequence into an intra-flow representation, and Stage 2 contextualises the resulting flow sequence before classifying the target flow.

4.1 Architectural Overview

Flow i is represented by a selected and padded packet-feature matrix $\mathbf{X}_i \in \mathbb{R}^{L \times d_p}$, a packet inter-arrival tensor $\mathbf{u}_i \in \mathbb{R}^L$, a validity mask $\mathbf{m}_i \in \{0, 1\}^L$, a statistical flow vector $\mathbf{s}_i \in \mathbb{R}^{d_f}$, and a binary target $y_i \in \{0, 1\}$. The principal configuration uses $L = 128$. Stage 1 computes

$$\left(\ell_i^{(1)}, \mathbf{z}_i^{(1)} \right) = F_{\theta_1}(\mathbf{X}_i, \mathbf{u}_i, \mathbf{m}_i, \mathbf{s}_i), \quad (4.1)$$

Here, $\ell_i^{(1)} \in \mathbb{R}^2$ denotes the Stage 1 logits, and $\mathbf{z}_i^{(1)} \in \mathbb{R}^d$ is the fused representation supplied to Stage 2. The principal model sets $d = 128$.

Once all Stage 1 embeddings are available, flows are stably ordered by start time and flow identifier. Given relation r and target i , the bounded causal input to Stage 2 is

$$\mathbf{Z}_i^{(r)} = \left[\mathbf{z}_{j_1}^{(1)}, \dots, \mathbf{z}_{j_{K_i}}^{(1)}, \mathbf{z}_i^{(1)} \right], \quad j_k < i, \quad K_i \leq W - 1, \quad (4.2)$$

The current flow occupies the final valid position, and the principal setting uses $W = 128$ tokens including that target. Stage 2 computes

$$\ell_i^{(2,r)} = G_{\theta_2}(\mathbf{Z}_i^{(r)}, \boldsymbol{\mu}_i), \quad (4.3)$$

where left padding is excluded by $\boldsymbol{\mu}_i$. Applying softmax to $\ell_i^{(2,r)}$ yields the final class-1 probability.

As indicated in Figure 4.2, labels and Suricata alert fields serve only as targets and are absent from both encoders.

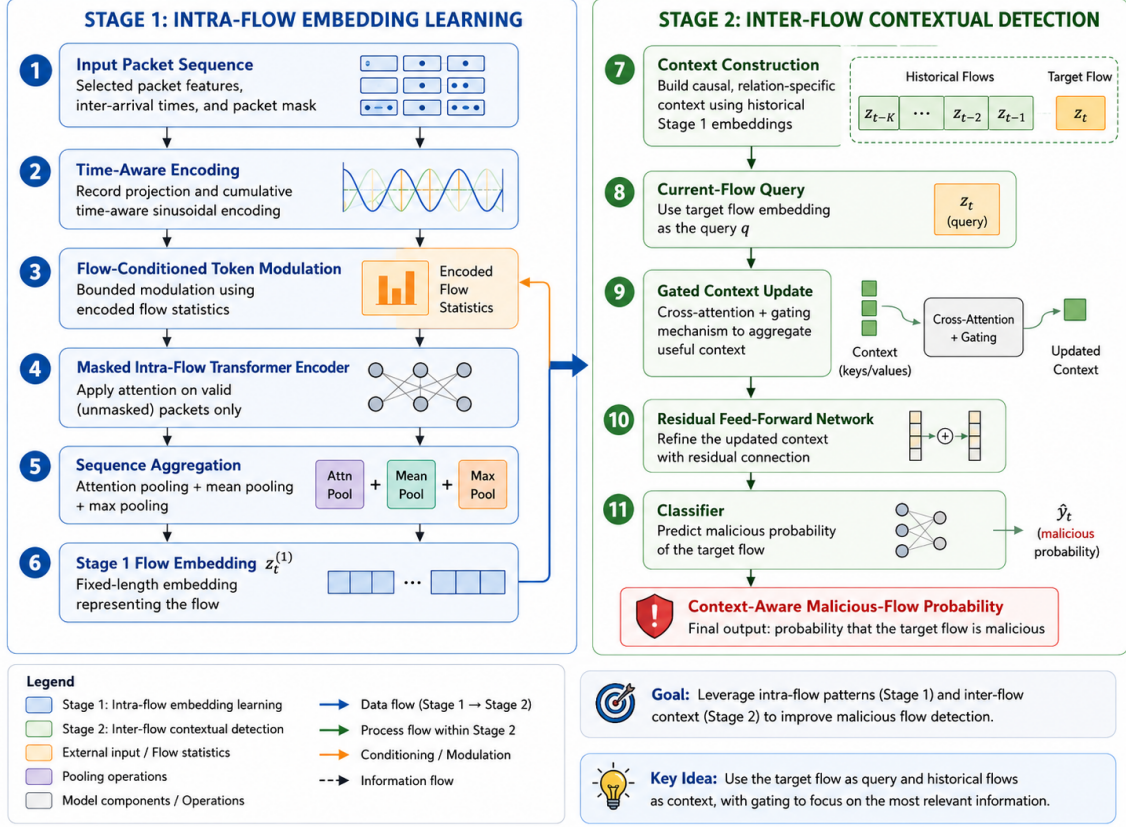


FIGURE 4.2: Detailed architecture of the proposed two-stage detector. The statistical branch conditions Stage 1 packet tokens, while Stage 2 uses the target flow as the query and historical flows as keys and values.

4.2 Stage 1: Intra-Flow Time-Aware Representation

A variable-length packet sequence is reduced by Stage 1 to one flow embedding. The principal processing path comprises record projection, time-aware sinusoidal encoding, flow-conditioned token modulation, pre-normalised Transformer layers, hybrid mask-aware pooling, and a two-class head.

4.2.1 Packet-Record Projection

At packet position t , the preprocessed vector $\mathbf{x}_{i,t} \in \mathbb{R}^{d_p}$ is mapped into the model space:

$$\mathbf{e}_{i,t} = \text{LN}(W_p \mathbf{x}_{i,t} + \mathbf{b}_p), \quad \mathbf{e}_{i,t} \in \mathbb{R}^d. \quad (4.4)$$

An MLP projection is supported, although the principal configuration applies a single linear map followed by layer normalisation. As specified in Chapter 3, categorical fields have already been one-hot encoded, numerical variables transformed and robustly scaled, and binary indicators retained in $\{0, 1\}$.

4.2.2 Cumulative Time-Aware Encoding

Sinusoidal positional encoding captures ordinal packet order but assigns the same adjacency to gaps of microseconds and seconds. Stage 1 augments position with cumulative elapsed

time. Preprocessing stores

$$u_{i,t} = \log(1 + \Delta\tau_{i,t}), \quad (4.5)$$

where $\Delta\tau_{i,t}$ is the nonnegative microsecond gap from the preceding packet in the same bidirectional flow. The model reconstructs and masks this interval as

$$\widehat{\Delta\tau}_{i,t} = m_{i,t} \max(0, \exp(u_{i,t}) - 1). \quad (4.6)$$

Cumulative time and its smoothed logarithmic form are then

$$c_{i,t} = \sum_{k=1}^t \widehat{\Delta\tau}_{i,k}, \quad (4.7)$$

$$p_{i,t} = \log(1 + c_{i,t} + \alpha), \quad \alpha = 10^{-7}. \quad (4.8)$$

With zero-based position $q_t = t - 1$, order and time define the coordinate $r_{i,t} = q_t + p_{i,t}$. Channel pair j is encoded by

$$\text{TE}_{i,t,2j} = \sin\left(\frac{r_{i,t}}{10000^{2j/d}}\right), \quad (4.9)$$

$$\text{TE}_{i,t,2j+1} = \cos\left(\frac{r_{i,t}}{10000^{2j/d}}\right). \quad (4.10)$$

Padding positions are assigned zero encodings. A valid packet token becomes

$$\widetilde{\mathbf{e}}_{i,t} = \text{Dropout}(\mathbf{e}_{i,t} + m_{i,t} \text{TE}_{i,t}). \quad (4.11)$$

This formulation adapts the time-aware Transformer principle in GTID [14] to structured packet metadata without payload content. Two ablations isolate its coordinates: time-only encoding sets $r_{i,t} = p_{i,t}$, whereas position-only encoding sets $r_{i,t} = q_t$.

4.2.3 Flow-Statistics Encoder

A bottleneck encoder maps the separately preprocessed flow vector $\mathbf{s}_i$ to the packet-model dimension:

$$\mathbf{f}_i = \text{LN}_2(W_{f,2} \text{Dropout}(\text{GELU}(\text{LN}_1(W_{f,1}\mathbf{s}_i + \mathbf{b}_{f,1}))) + \mathbf{b}_{f,2}). \quad (4.12)$$

Its principal hidden and output dimensions are 32 and 128, respectively. Restricting the statistical branch in this way reduces the chance that a high-dimensional aggregate vector dominates evidence learned from packets.

4.2.4 Bounded Flow-Conditioned Token Modulation

Feature-wise linear modulation (FiLM) motivates the principal Stage 1 fusion mechanism: conditioning information generates affine parameters for individual features [41]. A small network derives from the flow representation

$$[\gamma_i; \beta_i] = G_f(\mathbf{f}_i), \quad \gamma_i, \beta_i \in \mathbb{R}^d. \quad (4.13)$$

The unbounded flow-conditioned residual for packet token t is

$$\mathbf{r}_{i,t}^{\text{raw}} = \eta(\gamma_i \odot \tilde{\mathbf{e}}_{i,t} + \beta_i), \quad \eta = \sigma(a), \quad a_0 = -5, \quad (4.14)$$

where $\odot$ is element-wise multiplication and the initial scale is $\sigma(-5) \approx 0.0067$. A second constraint bounds this residual relative to the packet-token norm. For the configured maximum ratio $\rho = 0.13$,

$$\lambda_{i,t} = \min \left(1, \frac{\rho \|\tilde{\mathbf{e}}_{i,t}\|_2}{\|\mathbf{r}_{i,t}^{\text{raw}}\|_2 + \epsilon} \right), \quad (4.15)$$

$$\mathbf{e}_{i,t}^F = \tilde{\mathbf{e}}_{i,t} + \lambda_{i,t} \mathbf{r}_{i,t}^{\text{raw}}. \quad (4.16)$$

Zero initialisation is used for the layer that produces γ_i and β_i . Training therefore starts from the packet-only pathway, after which a bounded statistical correction can be learned. Such a constraint is useful when aggregate features are informative yet potentially noisy or capture-specific.

Because the statistical vector is encoded separately and then modulates packet tokens before self-attention, the principal configuration constitutes *separate-branch conditional intermediate fusion*, rather than strict late fusion after pooling. Although a post-pooling gated alternative is implemented, the reported Scheme C results use the bounded token-FiLM operation in Equation 4.16.

4.2.5 Masked Transformer Encoder

Pre-normalised Transformer layers process the conditioned sequence $\mathbf{E}_i^F \in \mathbb{R}^{L \times d}$. At layer ℓ ,

$$\mathbf{A}_i^\ell = \text{MHA} \left(\text{LN}(\mathbf{H}_i^{\ell-1}), \text{LN}(\mathbf{H}_i^{\ell-1}), \text{LN}(\mathbf{H}_i^{\ell-1}); \mathbf{m}_i \right), \quad (4.17)$$

$$\overline{\mathbf{H}}_i^\ell = \mathbf{H}_i^{\ell-1} + \text{Dropout}(\mathbf{A}_i^\ell), \quad (4.18)$$

$$\mathbf{H}_i^\ell = \overline{\mathbf{H}}_i^\ell + \text{FFN} \left(\text{LN}(\overline{\mathbf{H}}_i^\ell) \right), \quad (4.19)$$

with $\mathbf{H}_i^0 = \mathbf{E}_i^F$. Scaled dot-product attention is applied in every head:

$$\text{Attn}(Q, K, V) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + M_i \right) V, \quad (4.20)$$

Here, M_i assigns negative infinity to padded keys. GELU is used in the feed-forward sublayer. The principal encoder contains two layers and four attention heads, with model dimension 128, feed-forward dimension 256, and dropout 0.15.

Stage 1 does not apply a causal mask, so every retained packet in the completed or truncated flow may attend to every other retained packet. The stage should therefore be interpreted as flow-level classification, not packet-by-packet early detection. Padding is excluded both as an attention key and during pooling.

4.2.6 Hybrid Mask-Aware Pooling

Three complementary summaries reduce the packet states. First, each valid token receives a learned attention score:

$$a_{i,t} = \mathbf{w}_{a,2}^\top \text{GELU}(W_{a,1} \mathbf{h}_{i,t} + \mathbf{b}_{a,1}) + b_{a,2}, \quad (4.21)$$

$$\omega_{i,t} = \frac{m_{i,t} \exp(a_{i,t})}{\sum_{k=1}^L m_{i,k} \exp(a_{i,k})}, \quad (4.22)$$

$$\mathbf{z}_i^{\text{att}} = \sum_{t=1}^L \omega_{i,t} \mathbf{h}_{i,t}. \quad (4.23)$$

Mask-aware mean and maximum summaries are

$$\mathbf{z}_i^{\text{mean}} = \frac{\sum_{t=1}^L m_{i,t} \mathbf{h}_{i,t}}{\max(1, \sum_{t=1}^L m_{i,t})}, \quad (4.24)$$

$$\mathbf{z}_i^{\text{max}} = \max_{t:m_{i,t}=1} \mathbf{h}_{i,t}. \quad (4.25)$$

All three summaries contribute to the Stage 1 embedding:

$$\mathbf{z}_i^{(1)} = \text{Dropout} \left(\text{GELU} \left(\text{LN} \left(W_o [\mathbf{z}_i^{\text{att}}; \mathbf{z}_i^{\text{mean}}; \mathbf{z}_i^{\text{max}}] + \mathbf{b}_o \right) \right) \right). \quad (4.26)$$

Attention pooling can concentrate on a few informative packets, the mean captures overall behaviour, and the maximum retains strong local activations. Projecting their concatenation back to dimension d gives Stage 2 a fixed-size vector for every flow.

Stage 1 classification logits are

$$\ell_i^{(1)} = h_1(\mathbf{z}_i^{(1)}), \quad \mathbf{p}_i^{(1)} = \text{softmax}(\ell_i^{(1)}). \quad (4.27)$$

Export occurs before the final two-class layer, preserving a multi-dimensional representation for contextual refinement instead of reducing the flow to one probability.

4.2.7 Stage 1 Fusion Variants

Table 4.1 defines three controlled variants. Unless stated otherwise, packet features, sequence policy, data split, Transformer capacity, optimisation procedure, and classifier are held constant across them.

TABLE 4.1: Stage 1 packet/flow integration variants.

Scheme	Input structure	Integration mechanism
A	Packet + repeated flow statistics	The transformed flow vector is concatenated to every packet before record projection.
B	Packet only	The flow-statistics branch is disabled; this isolates packet order and time-aware encoding.
C	Packet + separate flow branch	The flow vector is encoded once and conditionally modulates packet tokens through bounded token-FiLM. This is the principal proposed Stage 1 configuration.

4.3 Stage 2: Causal Inter-Flow Context Modelling

Stage 2 tests whether recent related behaviour changes the interpretation of a target flow. Its inputs are restricted to Stage 1 embeddings and context metadata; raw packet fields, aggregate flow features, and historical labels are not loaded.

4.3.1 Embedding Interface and Chronological Order

For each split, Stage 1 exports

$$\left(\text{flow_id}_i, y_i, \mathbf{z}_i^{(1)}, t_i^0, u_i, v_i, \rho_i\right), \quad (4.28)$$

where t_i^0 gives the start time, u_i and v_i identify source and destination, and ρ_i records split membership. Stage 2 verifies identifier uniqueness, concatenates the split files, and applies a stable sort on $(t_i^0, \text{flow_id}_i)$. Embeddings and metadata are reordered together to preserve alignment.

A deterministic loader without sampling is used for embedding export. The weighted training sampler draws with replacement and would otherwise repeat some flows while omitting others; it is therefore restricted to optimisation batches.

4.3.2 Relation-Specific Causal Contexts

Context construction proceeds once through the sorted flows in temporal order. Eligible earlier indices are retrieved before target i is added to any history. Table 4.2 defines the four relations considered.

TABLE 4.2: Stage 2 context relations.

Method	Eligibility condition for $j < i$	Interpretation
<code>time_only</code>	Always eligible	Most recent flows globally, regardless of end-point identity.
<code>source_host</code>	$u_j = u_i$	Earlier flows initiated by the same source identifier.
<code>destination_host</code>	$v_j = v_i$	Earlier flows directed to the same destination identifier.
<code>endpoint</code>	$\{u_j, v_j\} \cap \{u_i, v_i\} \neq \emptyset$	Earlier flows sharing at least one endpoint in either role.

For a given relation r , define

$$\mathcal{A}_i^{(r)} = \{j < i : R_r(i, j) = 1\}. \quad (4.29)$$

Split-policy filtering and deduplication are applied before retaining the latest $W - 1$ indices:

$$\mathcal{C}_i^{(r)} = \text{Recent}_{W-1} \left(\mathcal{A}_i^{(r)} \right). \quad (4.30)$$

Appending the current flow yields Equation 4.2. Histories are not updated until this sequence has been formed, preventing both self-inclusion and access to future items. Under the principal **online** policy, a validation target may retrieve earlier training flows; a test target may retrieve earlier training, validation, and test flows. Future flows and historical labels remain unavailable. This policy approximates a detector whose state persists through chronological deployment.

In this implementation, “causal” refers to stable flow-start order: every context index occurs before its target index. The stronger condition $t_j^{\text{end}} \leq t_i^0$ is not enforced during Stage 2 export. A long-lived overlapping flow can therefore precede the target even when its complete embedding would not have been available at the target’s start time. What is evaluated is thus start-ordered, label-free context, rather than a fully timed streaming system. Production eligibility should instead be based on flow completion or embedding availability.

The **time_only** relation denotes global chronological recency; it does not provide continuous inter-flow gaps to Stage 2. Explicit microsecond timing is confined to Stage 1, whereas Stage 2 represents relative age within the selected context.

Short sequences are padded on the left during collation, keeping the target at the final valid position:

$$[\text{PAD}, \dots, \text{PAD}, \mathbf{z}_{j_1}^{(1)}, \dots, \mathbf{z}_{j_{K_i}}^{(1)}, \mathbf{z}_i^{(1)}]. \quad (4.31)$$

A Boolean mask prevents the physical padding positions from affecting either positional encoding or attention.

4.3.3 Context-Age Positional Encoding

For a target with K_i historical tokens, age 0 is assigned to the target, age 1 to the most recent historical flow, and age K_i to the oldest retained item. Writing this index as $a_{i,k}$, Stage 2 adds the sinusoidal encoding

$$\text{PE}_{i,k,2j} = \sin \left(\frac{a_{i,k}}{10000^{2j/d}} \right), \quad (4.32)$$

$$\text{PE}_{i,k,2j+1} = \cos \left(\frac{a_{i,k}}{10000^{2j/d}} \right). \quad (4.33)$$

Because age is independent of absolute tensor position, additional left padding does not change it. Identical target-history orderings consequently receive identical encodings even when batch sequence lengths differ.

4.3.4 Target-Query Gated Multi-Head Attention

The principal Stage 2 model, `target_query_gated`, does not update every historical token through pairwise self-attention. Instead, the current flow supplies a single query over its eligible history. Let $\mathbf{c}_i$ denote the projected Stage 1 representation of the current flow and $\tilde{\mathbf{H}}_i \in \mathbb{R}^{K_i \times d}$ the age-encoded history. Attention head h computes

$$\mathbf{q}_i^h = W_Q^h \text{LN}(\mathbf{c}_i + \text{PE}(0)), \quad (4.34)$$

$$\mathbf{K}_i^h = \text{LN}(\tilde{\mathbf{H}}_i) W_K^h, \quad (4.35)$$

$$\mathbf{V}_i^h = \text{LN}(\tilde{\mathbf{H}}_i) W_V^h, \quad (4.36)$$

$$\mathbf{h}_i^h = \text{softmax} \left(\frac{\mathbf{q}_i^h (\mathbf{K}_i^h)^\top}{\sqrt{d_h}} + M_i \right) \mathbf{V}_i^h. \quad (4.37)$$

Concatenation and projection of the head outputs produce the historical summary $\mathbf{h}_i$. When the eligible history is empty, $\mathbf{h}_i$ is set explicitly to zero so that an artificial padding item cannot supply context.

Difference and multiplicative terms compare the current representation with its context summary:

$$\mathbf{r}_i = [\mathbf{c}_i; \mathbf{h}_i; \mathbf{c}_i - \mathbf{h}_i; \mathbf{c}_i \odot \mathbf{h}_i], \quad (4.38)$$

$$\mathbf{v}_i = [\mathbf{h}_i; \mathbf{c}_i - \mathbf{h}_i; \mathbf{c}_i \odot \mathbf{h}_i]. \quad (4.39)$$

A dimension-wise gate controls the subsequent context update:

$$\mathbf{g}_i = \sigma(G(\mathbf{r}_i)), \quad (4.40)$$

$$\mathbf{d}_i = U(\mathbf{v}_i), \quad (4.41)$$

$$\mathbf{f}_i = \text{LN}(\mathbf{c}_i + \mathbf{g}_i \odot \mathbf{d}_i). \quad (4.42)$$

If no history exists, $\mathbf{g}_i$ and $\mathbf{d}_i$ are both zero and the current-flow representation passes through unchanged. A residual feed-forward block then produces

$$\mathbf{z}_i^{(2)} = \text{LN}(\mathbf{f}_i + \text{FFN}(\mathbf{f}_i)), \quad (4.43)$$

$$\ell_i^{(2)} = h_2(\mathbf{z}_i^{(2)}). \quad (4.44)$$

Target-query attention preserves a direct path for the current flow and uses history as a learned correction. Since there is only one query, the attention-score matrix grows linearly rather than forming the $W \times W$ matrix required by full self-attention. Multi-head retrieval over a 128-flow window is therefore computationally practical.

4.4 Imbalance-Aware Objective and Stage Interface

4.4.1 Focal Objective and Controlled Sampling

Two-class softmax outputs and focal loss are used in both stages. For target y_i with assigned-class probability p_{i,y_i} ,

$$\mathcal{L}_{\text{focal}} = -\frac{1}{B} \sum_{i=1}^B \alpha_{y_i} (1 - p_{i,y_i})^\gamma \log p_{i,y_i}. \quad (4.45)$$

Originally introduced for severe foreground/background imbalance, focal loss down-weights examples that are already classified easily [42]. Table 5.2 reports the chosen focal exponent, class weights, and label-smoothing values.

A controlled weighted sampler additionally determines training batches. Given class counts n_0 and n_1 and desired expected positive share q , negative and positive examples receive

$$w_0 = \frac{1-q}{n_0}, \quad w_1 = \frac{q}{n_1}. \quad (4.46)$$

Sampling changes expected minority exposure but leaves both the underlying dataset and the loss definition intact. Chapter 5 specifies stage-specific target proportions and loader policies.

4.4.2 Sequential Stage Interface

After Stage 1 has been trained for flow classification, its validation-selected checkpoint is restored. A deterministic loader without sampling exports one $\mathbf{z}_i^{(1)}$ and the required context metadata for every flow. These embeddings remain fixed during Stage 2 training, and no Stage 2 gradient reaches the earlier encoder. The sequential interface consequently allows Stage 1 comparisons to vary intra-flow representation while all Stage 2 models reason over a common representation.

5 Experimental Methodology

Within a comparison block, the data manifest, fitted preprocessing state, optimisation budget, checkpoint criterion, threshold rule, and random seed remain fixed. Validation PR-AUC determines the checkpoint, and validation class-1 F1 determines the decision threshold. The untouched test partition is evaluated only after both choices have been fixed. This protocol limits temporal leakage and feedback from test results, two sources of optimistic NIDS estimates [2, 40].

5.1 Evaluation Manifests and Leakage Controls

Evaluation is organised around two chronological manifests. Comparisons of Stage 1 architectures, encodings, fusion schemes, sequence lengths, and selection policies use 35,410 test flows: 34,003 negative and 1,407 positive. The hierarchical Scheme C reference and every Stage 2 experiment instead use the complete 84,380-flow test partition, comprising 80,860 negative and 3,520 positive flows. Results from these manifests are not paired, even when their rounded values happen to be close; effect sizes are calculated only between models sharing the same manifest.

All models within a block receive identical flows, packet and flow tensors, partition assignments, and train-fitted preprocessing. Training data alone determine category vocabularies, clipping limits, scaling parameters, and sampler weights. Configured class

weights and sampling targets are also chosen without test feedback; neither validation nor test observations contribute to fitted transformations.

One frozen Stage 1 checkpoint supplies all Stage 2 inputs. Under online context construction, only embeddings earlier in stable start-time order are eligible. Validation targets may consult preceding training flows, while test targets may consult preceding flows from training, validation, or testing. A history is updated after the current context has been assembled, and it never exposes future embeddings or past labels. Embedding files, meta-data ordering, context indices, and masks are reused across Stage 2 architectures, isolating the encoder or relation under study from context availability. Section 4.3.2 distinguishes this start-ordered condition from strict availability at flow-completion time.

5.2 Experimental Matrix

Table 5.1 distinguishes experiments on the packet encoder from those evaluating the complete hierarchy.

TABLE 5.1: Experimental matrix. Only the factor listed in the final column is changed within each experiment group.

Group	Stage	Fixed reference	Compared alternatives	Varied factor
S1-E1	1	Stage 1 manifest	Flow MLP; packet sequence models; proposed Stage 1	Encoder family
S1-E2	1	Scheme B, $L = 128$, head	Both, position-only, time-only, no encoding	Temporal/order encoding
S1-E3	1	Both encodings, $L = 128$, head	Schemes A, B, C	Flow-statistics integration
S1-E4	1	Scheme B, head	$L = 8, 16, 32, 64, 128, 256$	Packet budget
S1-E5	1	Scheme B, $L = 128$	Head, head-tail, random	Truncation policy
S2-E1	2	flow manifest; same frozen embeddings	No-context MLP; Transformer; LSTM; GRU; CNN+LSTM; proposed	Context encoder
S2-E2	2	Proposed model, $W = 128$	Time-only, source-host, destination-host, endpoint	Context relation
S2-E3	2	Proposed model, source-host	$W = 16, 32, 64, 128, 256$	Context capacity

5.2.1 Stage 1 Baselines

Stage 1 is evaluated against the following baselines:

- **Flow-level MLP.** A multilayer perceptron consumes only the aggregated statistical flow vector. It provides the direct reference for testing whether packet-level sequence information adds value beyond conventional flow aggregation.
- **CNN1D.** Parallel one-dimensional convolutions with kernel sizes 3, 5, and 7 capture local packet motifs, followed by global pooling and classification. A second variant concatenates the log inter-arrival feature to each packet record. The design follows the general use of multi-kernel convolution for sequence classification [43].
- **LSTM and BiLSTM.** Two-layer recurrent encoders with hidden width 128 and mean pooling provide unidirectional and bidirectional sequence references. The LSTM-with-time variant concatenates the stored log inter-arrival value to each packet vector [44].
- **GRU.** A two-layer, width-128 GRU with mean pooling tests whether a lighter gated recurrent encoder is sufficient [45].

Packet tensors, masks, the chronological partition, focal objective, sampler, optimiser, training budget, and model-selection protocol are shared by all Stage 1 baselines. Parameter counts are not artificially matched. Instead, trainable parameters and runtime are reported beside predictive metrics to expose performance–efficiency trade-offs.

5.2.2 Stage 1 Ablations and Sensitivity Tests

Four variants separate the effects of order and elapsed time: (i) joint positional and time-aware encoding, (ii) position only, (iii) time only, and (iv) neither coordinate. Packet features and Transformer capacity are unchanged. This control matters because appending an interval to a feature vector is not equivalent to using elapsed time as the coordinate of a sinusoidal sequence representation.

The three schemes in Table 4.1 assess how flow statistics should be incorporated. Scheme A repeats the aggregate vector at every packet position; Scheme B removes it; Scheme C applies bounded token-FiLM conditioning from a separate branch. Packet-encoder and truncation experiments use Scheme B to isolate sequence modelling. Scheme C constitutes the complete Stage 1 model and provides embeddings for Stage 2.

Sequence-length sensitivity is tested at $L \in \{8, 16, 32, 64, 128, 256\}$ with `head` selection. A separate experiment holds $L = 128$ and contrasts `head`, `head_tail`, and deterministic random sampling. No other setting changes in either sweep.

5.2.3 Stage 2 Baselines and Context Ablations

Five controls are used to assess the target-query gated attention model:

- **No-context MLP** classifies only the current Stage 1 embedding and provides a minimal current-flow-only lower bound. Because it is not capacity matched to the contextual encoders, it is not by itself a causal estimate of the value of history.
- **Vanilla Transformer** applies full self-attention to the same context sequence and classifies its last valid token [12].
- **LSTM** and **GRU** replace attention with recurrent context encoders while preserving the Stage 1 inputs, context window, masks, and classifier protocol [44, 45].
- **CNN+LSTM** applies parallel temporal convolutions with kernel sizes 3, 5, and 7 before a two-layer LSTM, combining local pattern extraction with recurrent context modelling [43, 44].

During architecture comparison, each contextual baseline receives the same `source_host` histories at $W = 128$, with the current flow occupying the final valid position. Since the target consumes one position, at most 127 historical flows are available. The no-context control keeps the target embedding and discards the preceding history.

Relation sensitivity holds the proposed encoder and $W = 128$ fixed while varying eligibility among `time_only`, `source_host`, `destination_host`, and `endpoint`. Window sensitivity instead fixes `source_host` and evaluates $W \in \{16, 32, 64, 128, 256\}$. Run summaries contain predictive metrics and confusion matrices but do not retain target-level context occupancy or age. Interpretation of the window sweep therefore acknowledges that missing diagnostic information.

5.3 Training, Model Selection, and Thresholding

5.3.1 Principal Training Configuration

Principal architecture and optimisation settings are listed in Table 5.2. Both stages use AdamW, which decouples weight decay from adaptive gradient updates [46], together with focal loss to downweight easy majority-class examples [42].

TABLE 5.2: Principal optimisation configuration. Values changed in a named ablation are the only exceptions.

Setting	Stage 1	Stage 2
Architecture and input	Scheme C; $L = 128$; head selection; $d = 128$	Target-query gated attention; source-host; $W = 128$; $d = 128$
Batch size	128	128
Optimiser	AdamW	AdamW
Learning rate	1×10^{-4}	3×10^{-5}
Weight decay	1×10^{-4}	3×10^{-4}
Loss	Focal, $\gamma = 1$, $\alpha = [1, 1.25]$, no label smoothing	Focal, $\gamma = 1$, $\alpha = [1, 1.01]$, no label smoothing
Sampler target	10% class-1 examples	5% class-1 examples
Maximum epochs	500	500
Early-stopping patience	25 epochs	15 epochs
Checkpoint criterion	Validation PR-AUC	Validation PR-AUC

Weighted sampling with replacement is restricted to training mini-batches. Validation, test, and embedding-export loaders preserve exactly one row per flow and do not sample, preventing duplicated embeddings or changes to the empirical evaluation distribution. CUDA runs use automatic mixed precision. Stage 1 lowers its learning rate after a plateau in validation PR-AUC, whereas Stage 2 follows five warm-up epochs with cosine annealing.

5.3.2 Checkpoint and Decision-Threshold Selection

Checkpoint and decision-threshold selection are separate operations applied uniformly across experiments. First, the checkpoint with the highest validation PR-AUC is restored. Validation predictions from that checkpoint then determine the class-1 threshold:

$$\delta^* = \arg \max_{\delta \in \Delta} F_{1,1}^{\text{val}}(\delta). \quad (5.1)$$

In Stage 1, Δ contains thresholds induced by the validation precision–recall curve; Stage 2 searches 199 equally spaced values between 0.01 and 0.99. Once selected, δ^* is fixed and transferred to the test partition without adjustment. Test observations play no role in checkpoint, threshold, or hyperparameter selection.

5.4 Evaluation Metrics

Metric calculations follow scikit-learn definitions [47]. The alert-derived positive class is label 1 and the negative class label 0. Confusion matrices use the fixed orientation

$$\mathbf{C} = \begin{bmatrix} \text{TN} & \text{FP} \\ \text{FN} & \text{TP} \end{bmatrix}, \quad (5.2)$$

with true labels on rows and predictions on columns. Fixing this convention removes ambiguity when matrices are transferred between software packages.

5.4.1 Class-Wise and Macro-Averaged Measures

For $c \in \{0, 1\}$, class-wise precision, recall, and F1 are

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad (5.3)$$

$$R_c = \frac{TP_c}{TP_c + FN_c}, \quad (5.4)$$

$$F_{1,c} = \frac{2P_cR_c}{P_c + R_c}. \quad (5.5)$$

Undefined ratios take the value zero, consistent with the implementation's `zero_division=0` setting. The macro-averaged metrics are

$$P_{\text{macro}} = \frac{P_0 + P_1}{2}, \quad (5.6)$$

$$R_{\text{macro}} = \frac{R_0 + R_1}{2}, \quad (5.7)$$

$$F_{1,\text{macro}} = \frac{F_{1,0} + F_{1,1}}{2}. \quad (5.8)$$

Macro-F1 is the primary F1 measure and is abbreviated to “F1” in high-level tables. It assigns equal weight to the negative and positive classes despite their unequal prevalence; macro-precision and macro-recall are interpreted analogously. To reveal error changes hidden by macro averaging, P_1 , R_1 , and $F_{1,1}$ are reported alongside these summaries.

Accuracy and weighted averages remain secondary: an always-negative rule would score approximately 96.055% accuracy on the raw data while detecting no positive flow. Neither high accuracy nor weighted F1 alone is therefore sufficient evidence of effective intrusion detection.

5.4.2 False-Positive Rate and Operational Error Counts

False-positive rate is defined as

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}}, \quad (5.9)$$

and false-negative rate as

$$\text{FNR} = \frac{\text{FN}}{\text{FN} + \text{TP}} = 1 - R_1. \quad (5.10)$$

Even a low FPR can generate substantial alert volume in a high-throughput network. It is therefore considered together with absolute FP and FN counts. At the evaluated operating point, a test partition with N_0 negative flows produces an expected $N_0 \times \text{FPR}$ false alerts.

5.4.3 Threshold-Independent Ranking Measures

ROC-AUC measures ranking through the true-positive rate over the range of false-positive rates [48]. Under strong class imbalance, however, a large negative population can coexist with a high ROC-AUC. PR-AUC concentrates on retrieval of the positive class and consequently receives greater emphasis here [39].

The reported `pr_auc` field is produced by scikit-learn’s `average_precision_score`. Thus, it denotes average precision (AP), rather than a trapezoidal area under linearly interpolated precision–recall points:

$$\text{AP} = \sum_k (R_k - R_{k-1}) P_k, \quad (5.11)$$

where P_k and R_k are evaluated at successive score thresholds. Every quantity labelled PR-AUC in the results therefore corresponds to AP. Since random ranking has expected AP near positive prevalence, observed values should be interpreted against the class-1 share of the relevant test manifest.

5.5 Efficiency and Resource Measurement

Experiments ran in Google Colaboratory Pro+ on an NVIDIA L4 GPU with 24 GB memory, using a company-funded allocation of 600 compute units [49, 50]. Those units regulate access to hosted resources and do not quantify a fixed amount of GPU computation. Runtime is compared only within the same stage because Stage 1 and Stage 2 differ in data loading, early stopping, and execution path.

Trainable parameters are counted directly from tensors that require gradients. Stage 1 wall time includes fitting, validation, and checkpointing. It is treated as a descriptive single-run cost because early stopping and variation in the hosted environment affect duration. For Stage 2 timing, gradients are disabled and the model is placed in evaluation mode. Per-flow latency is calculated as

$$t_{\text{flow}} = \frac{T_{\text{test}}}{N_{\text{test}}}, \quad Q = \frac{N_{\text{test}}}{T_{\text{test}}}, \quad (5.12)$$

where Q denotes flows processed per second. Timing starts after Stage 1 embeddings and context indices have been prepared, so it measures model evaluation rather than the complete path from PCAP to alert.

TABLE 5.3: Execution environment for the reported experiment blocks.

Component	Recorded environment
Hosted platform	Google Colaboratory Pro+
Purchased allocation	600 purchased compute units
Accelerator	NVIDIA L4 (24 GB)
Host processor and RAM	Google Colab hosted CPU; exact model not retained in the run metadata
Operating environment	Google Colab hosted Linux runtime
Learning software	PyTorch and scikit-learn on CUDA; exact version snapshot not retained
Traffic extractor	Suricata with the custom output module; exact version and revision not retained

Available run metadata do not retain the exact CPU, software snapshot, Suricata version, ruleset snapshot, configuration, custom-module revision, or PCAP checksum. Exact reproduction of runtime and label generation is consequently limited, and the missing provenance is identified as unavailable rather than reconstructed after the fact.

6 Results and Discussion

Unless explicitly qualified, F1, precision, and recall refer to macro averages; class-1 quantities carry a separate label. Discussion centres on macro-F1, PR-AUC, class-1 performance, FP, FN, and FPR because the majority class dominates accuracy. Numerical differences are interpreted only when the models share a test manifest.

6.1 Stage 1: Intra-Flow Representation

6.1.1 Architecture and Temporal-Encoding Effects

Stage 1 model families and encoding ablations are contrasted on a common flow test set in Figure 6.1; their parameter counts and training times appear in Table 6.1.

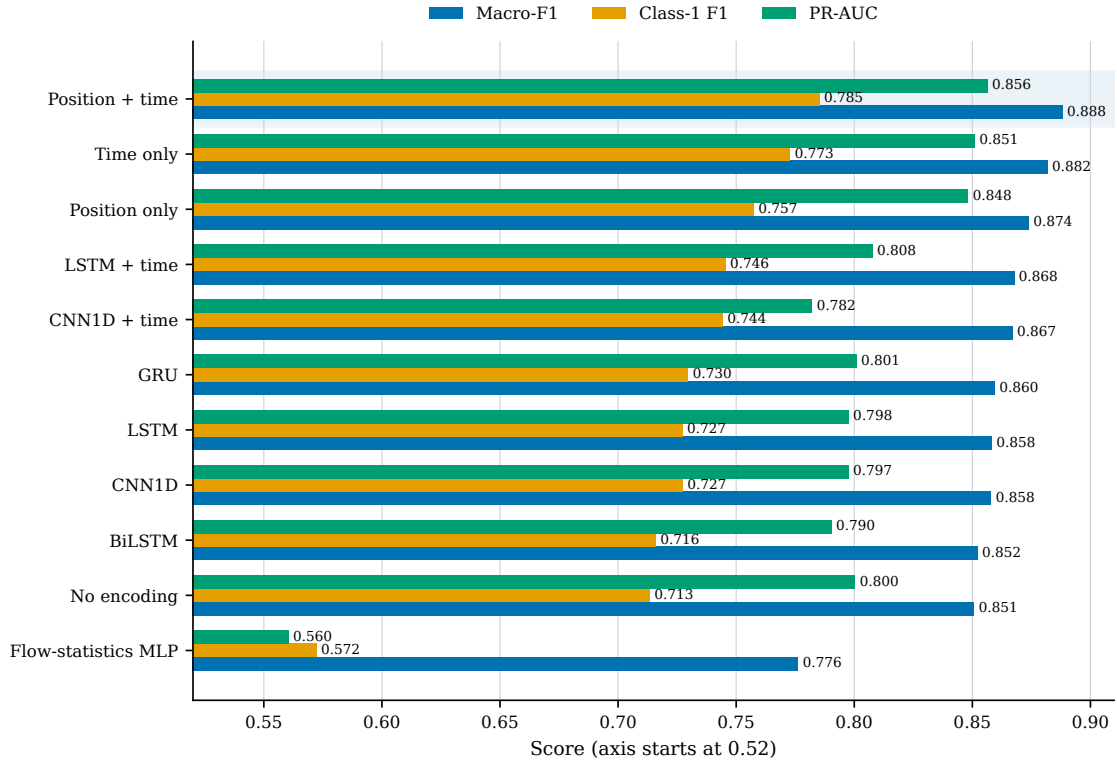


FIGURE 6.1: Stage 1 architecture and encoding comparison on the common flow test manifest. The horizontal axis begins at 0.52, and absolute values are printed at the end of each bar.

Among these models, position+time gives the Transformer the best overall balance: macro-F1 reaches 0.8882 and PR-AUC 0.8564. LSTM+time is the strongest non-Transformer alternative by macro-F1, yet the Transformer improves this score by 2.05 percentage points. At the selected operating point, 95 additional positive flows are recovered at the cost of 10 more false positives. The gain therefore reflects a materially different error balance rather than a change in a summary score alone.

Encoding ablations separate two meanings of sequence location. Position records where a packet occurs, whereas cumulative time distinguishes a rapid burst from an identical order spread over a longer interval. Time-only encoding exceeds position-only, and combining

them adds another 0.64 percentage points of macro-F1. Both FP and FN are also lower than under no encoding. These changes support order and elapsed time as complementary, rather than interchangeable, descriptions of a flow.

The lowest FPR, 0.8293%, belongs to the GRU, but that operating point misses 437 positive flows. Position+time introduces only 14 additional FP and removes 131 FN relative to the GRU. A low false-alert rate is therefore not sufficient by itself: conservative thresholds can suppress FP by transferring error to the positive class.

The flow-statistics MLP provides the direct aggregation-only control. It obtains macro-F1 0.7762 and PR-AUC 0.5604 on the same test composition, compared with 0.8882 and 0.8564 for position+time. Its 853 FP and 501 FN fall to 296 and 306 when packet dynamics are modelled. Duration, count, and inter-arrival summaries therefore retain useful global information but fail to reconstruct all of the local order and timing present in the packet sequence.

Operating-point errors are shown in Figure 6.2. Because the negative examples are shared, FP bar height is directly proportional to FPR.

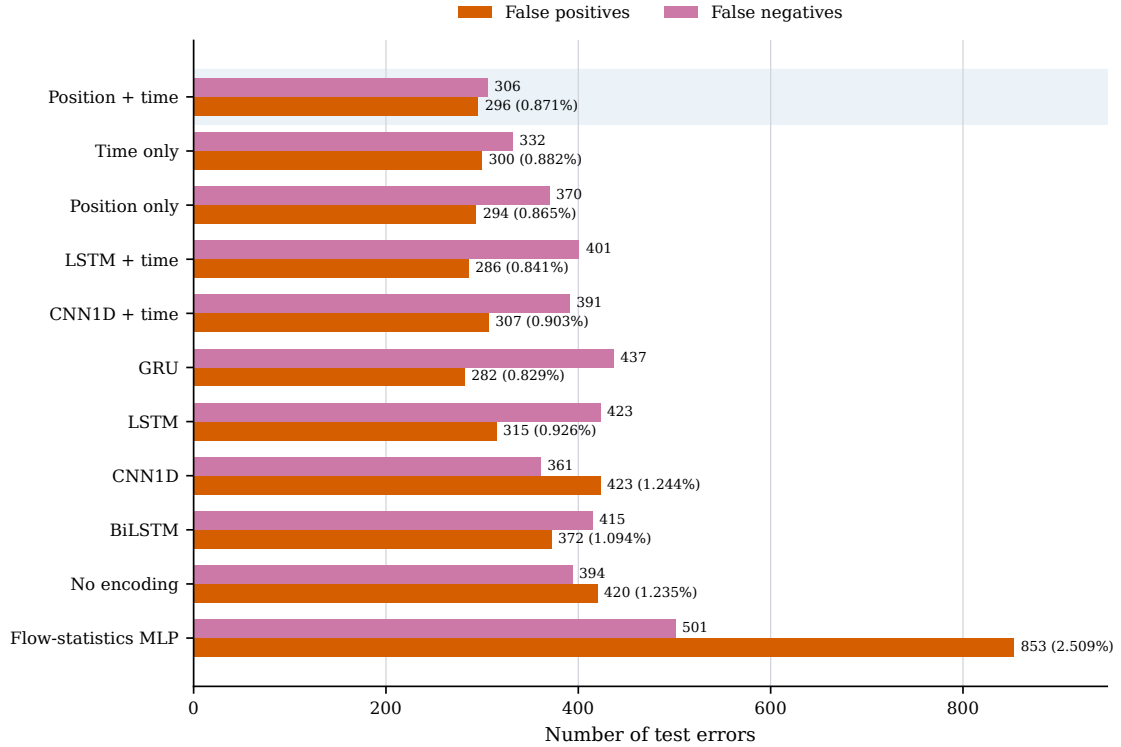


FIGURE 6.2: Stage 1 false-positive and false-negative counts on the common flow test manifest. FPR is printed beside each FP count.

LSTM also benefits from explicit time, with reductions in both error types. For CNN, however, the effect is mixed: the selected operating point improves, while PR-AUC declines and FN rises. This disagreement is informative. Thresholded F1 characterises one decision point, whereas PR-AUC evaluates ranking over thresholds. Timing is therefore not beneficial merely because it is present; its representation within the encoder matters, and an appended interval feature is not equivalent to an elapsed-time sequence coordinate.

TABLE 6.1: Stage 1 parameter counts and single-run training times measured on an NVIDIA L4 GPU in Google Colab.

Model	Trainable parameters	Training time (s)	Training time (min)
Position+time Transformer	344,963	4,337.6	72.29
Time-only Transformer	344,963	2,250.5	37.51
Position-only Transformer	344,963	1,994.7	33.25
LSTM+time	217,346	3,556.3	59.27
CNN1D+time	70,274	1,162.4	19.37
GRU	162,690	3,684.4	61.41
LSTM	216,834	3,381.2	56.35
CNN1D	68,354	1,591.0	26.52
BiLSTM	564,738	2,625.7	43.76
No-encoding Transformer	344,963	2,213.9	36.90

No MLP efficiency entry is shown in Table 6.1, as its parameter count and training time were not retained.

CNN1D+time records the shortest Stage 1 training time, 1,162.4 seconds; position+time requires 4,337.6 seconds. Despite the BiLSTM’s larger parameter count, the proposed encoder is the most expensive run observed. Each value was collected on an NVIDIA L4 GPU, but it includes early stopping, data loading, and variation in the hosted runtime. These measurements describe total training cost, not architecture-only throughput.

6.1.2 Packet and Flow-Statistics Integration

The fusion comparison in Figure 6.3 uses one shared flow manifest. Scheme A tiles the statistical vector across packet tokens, Scheme B retains packet inputs alone, and Scheme C conditions packet representations through bounded token-FiLM from a separate branch.

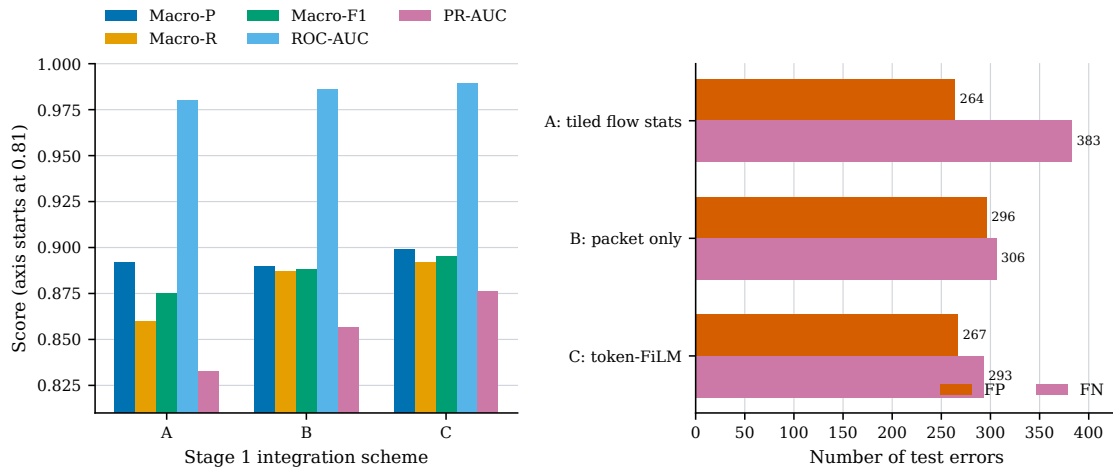


FIGURE 6.3: Controlled Stage 1 integration comparison on the common flow manifest. Left: macro metrics and threshold-independent ranking metrics. Right: false-positive and false-negative counts. The score axis is truncated and explicitly labelled.

Removing the tiled vector in Scheme B raises macro-F1 by 1.30 percentage points

and PR-AUC by 2.38 points over Scheme A, although Scheme A retains a small macro-precision advantage. Repetition of one aggregate vector at every position may therefore interfere with distinctions among packet tokens. Scheme C strengthens this interpretation: flow statistics appear more useful when encoded through a dedicated pathway than when copied into each packet position.

Scheme C is strongest overall, with macro-F1 0.8955, PR-AUC 0.8763, 267 FP, and 293 FN. Both error counts improve over packet-only Scheme B, indicating that aggregate statistics add information when they condition rather than replace packet evidence. Attribution must nevertheless remain at the level of the integrated design. Scheme C introduces both extra parameters and a separate statistical pathway, so the present ablation cannot assign the entire gain specifically to token-FiLM. A parameter-matched concatenation control would be needed for that narrower claim.

6.1.3 Sequence Length and Packet Selection

Figure 6.4 presents both sensitivity analyses: sequence length is traced in the line panel, and selection policies are compared through grouped precision and recall bars.

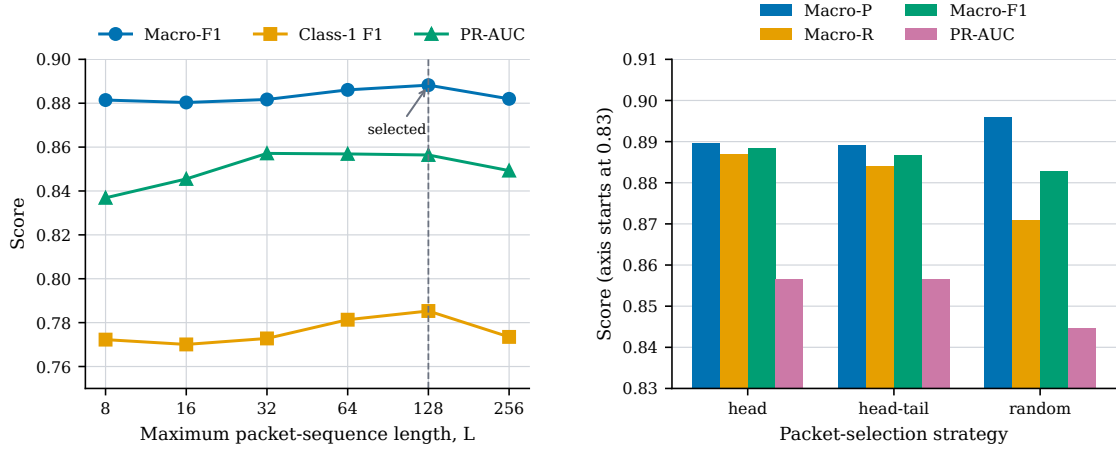


FIGURE 6.4: Stage 1 sensitivity analysis. Left: maximum packet-sequence length under head selection. Right: packet-selection strategy at $L = 128$. Dashed lines identify the selected sequence length. Truncated axes are explicitly labelled.

Longer sequences do not produce monotonic improvement. Ranking is weaker when only a few early packets are available, implying that those packets are not always sufficient for discrimination. Performance peaks at $L = 128$ by macro-F1 and overall error balance. Increasing the budget to 256 adds 46 FP and lowers both macro-F1 and PR-AUC; additional packets can therefore add volume without adding useful evidence.

Aggregate sweep results do not identify the mechanism behind this peak. Changing L affects long truncated flows but leaves sufficiently short flows unchanged, so dilution by later packets cannot be separated from flow-length composition or optimisation effects. Such an explanation would require length-stratified evaluation. Consequently, $L = 128$ is treated as the best observed budget for this capture, not a universal horizon.

At $L = 128$, the macro-F1 difference between head and head-tail is only 0.16 percentage points. With one seed, this margin is treated as practically inconclusive. Random selection shifts the detector towards lower FPR and more FN. Head remains the principal policy because it is deterministic, provides the best observed macro-F1 and attack-class recall,

and preserves a contiguous early prefix without waiting for the flow’s final packets. Its small advantage over head-tail is not interpreted as a robust ordering.

6.2 Stage 2: Inter-Flow Context

6.2.1 Gain over Independent Flow Classification

The principal cross-stage comparison appears in Table 6.2. Both rows share the same test flows: one classifies each Scheme C embedding independently, and the other augments that frozen representation with causal source-host history.

TABLE 6.2: Independent Stage 1 classification and the complete hierarchical system on the common full-data manifest.

Model	P_{macro}	R_{macro}	$F_{1,\text{macro}}$	$F_{1,1}$	ROC-AUC	PR-AUC	FPR (%)
Stage 1 Scheme C	0.8990	0.8919	0.8954	0.7994	0.9890	0.8763	0.8249
Stage 2 source-host	0.9328	0.9224	0.9275	0.8610	0.9938	0.9345	0.5429

Adding Stage 2 raises macro-F1 by 3.21 percentage points and PR-AUC by 5.82 points relative to independent Scheme C classification. The contextual system removes 228 FP and 205 FN, a relative FPR reduction of approximately 34.2%. Since both error types decline, the gain is not an accuracy artefact or a simple shift towards one class. This comparison concerns the hierarchy as an integrated system. Subsequent architecture controls test simpler encoders, but the capacity mismatch of the no-context head prevents the full difference from being attributed solely to historical evidence.

6.2.2 Which Relation Defines Useful Context?

Figure 6.5 links ranking quality to operating behaviour. Macro-F1 and PR-AUC are shown on the left; class-1 recall and FPR locate each relation on the right.

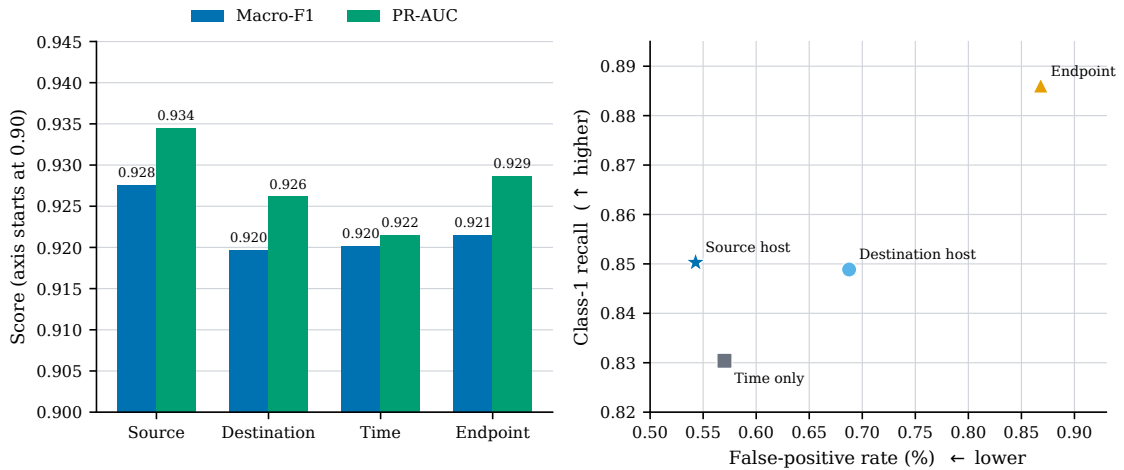


FIGURE 6.5: Stage 2 context-relation ablation at $W = 128$. Left: macro-F1 and PR-AUC. Right: class-1 recall versus FPR, where the desirable region is the upper-left corner.

Source-host history yields the strongest balance: macro-F1 is 0.9275, PR-AUC 0.9345, and FPR is the lowest among the relations. Repeated outbound activity from one initiator may preserve evidence of scanning, retries, or coordinated connections. Global recency, by comparison, mixes unrelated flows, while destination history may combine many benign clients of a widely used service. This interpretation agrees with relational and window-level findings in prior NIDS work [27, 30], but it remains a mechanism hypothesis because binary labels do not identify the attack families responsible for the gain.

Endpoint context operates at a different point: fewer positives are missed, but more false alerts are produced. Matching either endpoint in either role can retrieve lateral or bidirectional activity, while also admitting a more heterogeneous neighbourhood. It may be appropriate when FN carry a higher operational cost; under the balanced criterion used here, source-host remains preferable. Time-only context ranks flows less effectively and produces more FN, suggesting that chronological proximity without an entity relation is insufficiently selective for this capture.

6.2.3 How Much Context Is Useful?

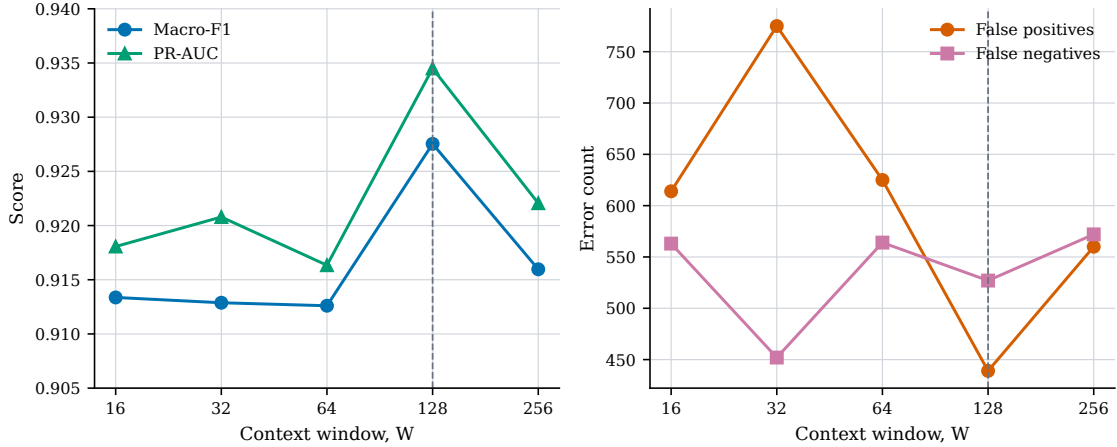


FIGURE 6.6: Source-host context-window sensitivity. Left: macro-F1 and PR-AUC. Right: false-positive and false-negative counts. The dashed line marks the selected window $W = 128$.

Context capacity is also non-monotonic in Figure 6.6. A 32-flow window maximises macro-recall, but its 775 FP show the cost of that sensitivity. At 128 flows, macro-F1 reaches 0.9275, PR-AUC reaches 0.9345, and FPR falls to its minimum of 0.5429%, giving the strongest joint outcome. Both FP and FN rise when the window expands to 256. This pattern is compatible with a finite useful horizon: short histories may omit relevant activity, while increasingly old flows from the same source may have little connection to the current event.

Nominal window size alone cannot explain this response. The run summaries do not retain target-level counts or age distributions for valid context items, so older evidence, padding frequency, and unequal history coverage cannot be disentangled. Performance stratified by realised context length and elapsed span would be needed to test the horizon explanation directly. Within the present design, $W = 128$ is therefore an empirical operating point for this capture, not a universal optimum.

6.2.4 Stage 2 Architecture Comparison

Architecture comparison fixes the Stage 1 embeddings, source-host relation, $W = 128$, online policy, objective, optimiser, sampler, checkpoint criterion, and validation threshold search. Only the context encoder and dimensions specific to that encoder vary. Six architectures are displayed in Figure 6.7; FP and FN are shown separately in the second panel so that aggregate F1 does not conceal their trade-off.

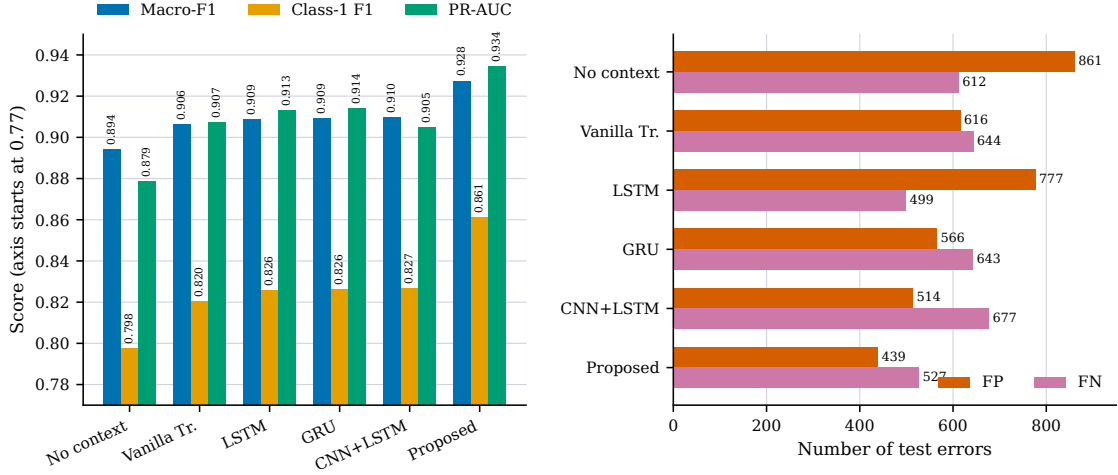


FIGURE 6.7: Stage 2 architecture comparison on the common flow test manifest. Left: macro-F1, class-1 F1, and PR-AUC. Right: threshold-dependent false-positive and false-negative counts. Truncated score axes are labelled.

Target-query attention produces the best combined result, with macro-F1 0.9275, class-1 F1 0.8610, and PR-AUC 0.9345. The nearest baseline values are CNN+LSTM’s 0.9097 macro-F1 and GRU’s 0.9141 PR-AUC. LSTM has slightly higher macro-recall, yet that advantage is accompanied by weaker threshold-independent ranking and a less favourable false-alert balance.

Confusion matrices make these differences operationally concrete. Against the vanilla Transformer, target-query attention eliminates 177 FP and 117 FN on the same flows. LSTM recovers 28 more positives than the proposed model but generates 338 additional false alerts. The macro-F1 gain over vanilla attention is therefore not obtained by exchanging one error type for the other, and the precision advantage over the highest-recall recurrent baseline is substantial.

This contrast is consistent with the intended inductive bias. Vanilla self-attention updates the full context sequence and classifies its final valid state; the proposed encoder keeps the current flow as the query and gates the retrieved historical summary. Simultaneous improvement in macro-F1 and PR-AUC is compatible with stronger target-centred ranking of positive examples. It is not, however, a clean isolation of the query pattern, because other architectural choices and parameter counts also differ.

With 514 parameters, the no-context head is far smaller than the 283,522-parameter Stage 2 model. It should be read as a current-flow-only lower bound, not a capacity-matched ablation. Its result shows that the minimal independent head is insufficient, but does not identify how much of the full gap comes from history. A stricter control would preserve the target-query architecture and comparable capacity while masking every historical item.

6.2.5 Efficiency and Operational Trade-Offs

Parameter count and measured evaluation latency are related in Figure 6.8, with macro-F1 annotated for each architecture. A logarithmic parameter axis accommodates the range from 514 to more than 1.2 million parameters.

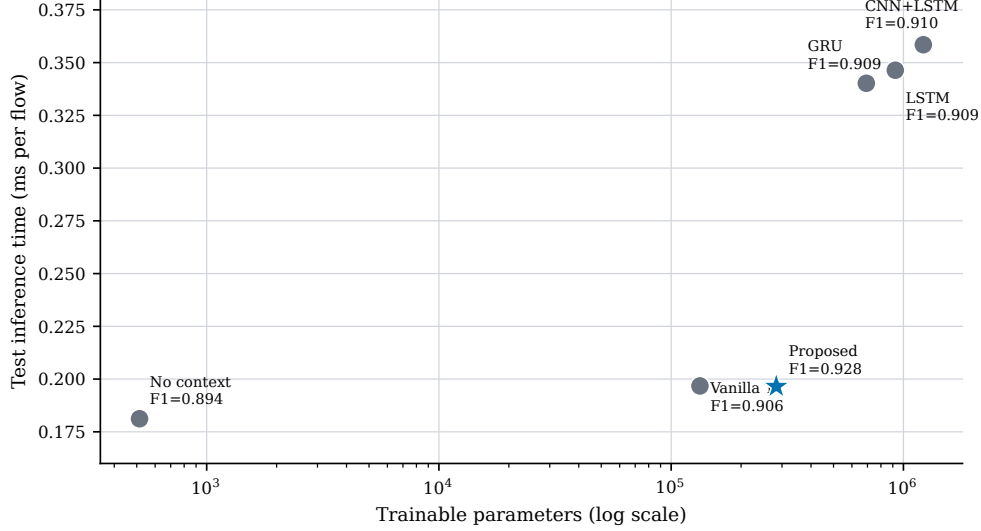


FIGURE 6.8: Stage 2 performance–efficiency trade-off. Latency covers model evaluation after Stage 1 embeddings and context indices have been built; it is not end-to-end PCAP-to-alert latency.

All profiles in Figure 6.8 were collected on an NVIDIA L4. The proposed model has 283,522 parameters and evaluates one flow in 0.1965 ms on average, equivalent to approximately 5,089 flows per second for the measured model path. Vanilla Transformer latency is 0.1967 ms. A difference of 0.0002 ms is too small to support a speed claim; the meaningful observation is latency parity accompanied by a better error balance.

Compared with GRU, LSTM, and CNN+LSTM, parameter count falls by 59.0%, 69.3%, and 76.7%, respectively; measured latency falls by 42.2%, 43.3%, and 45.2%. Unlike the negligible margin over vanilla attention, these differences indicate a genuine compactness and model-level latency advantage over recurrent alternatives. They are not estimates of deployed end-to-end throughput. Context indexing, Stage 1 inference, Suricata extraction, and PCAP processing are excluded, and one aggregate timing cannot characterise warm-up, run-to-run variation, batching effects, or tail latency.

6.3 Cross-Stage Discussion

Different losses of information are addressed at the two levels. Stage 1 restores order and elapsed time that aggregate vectors discard; Stage 2 introduces behavioural evidence absent from independent classification. Ablation results support both roles: position and time contribute complementary intra-flow information, and start-ordered source-host history improves the detector after its Stage 1 representation is fixed. In this respect, the hierarchy extends packet-aware systems such as GTID and modular flow encoders such as FlowTransformer with an explicitly evaluated cross-flow reasoning layer [14, 15].

With respect to RQ1, time-aware packet modelling exceeds the statistical MLP on a shared manifest, while fusion results show that aggregate measurements still help when used

for conditioning rather than replacement. The sequence-length sweep qualifies this result: useful representation depends on which packets are retained and encoded, not on unlimited token growth. For RQ2, Stage 2 lowers both FP and FN relative to independent Scheme C predictions. Relation and window experiments add a second qualification: context must be selective, and additional neighbouring flows are not automatically useful.

These findings accord with temporal sequence models and graph-based analyses that treat traffic records as dependent [18, 27, 28]. They do not establish source-host history as the best relation for every attack, nor can binary labels reveal which families benefit. A stronger cross-stage analysis would align individual errors, identify which Stage 1 FP and FN are corrected by Stage 2, and relate those transitions to relation type, realised context size, and context age.

Taken together, the evidence indicates that temporal structure within a flow and selective relations across flows remedy different deficiencies of aggregate, independent flow classification.

7 Conclusion and Future Work

7.1 Conclusions

RQ1: intra-flow representation. RQ1 is answered in favour of modelling packets as an ordered, time-aware sequence rather than relying only on aggregate flow statistics. Ordinal position and elapsed time provide distinct and complementary information about how a flow develops, while a separate statistical conditioning and fusion pathway further improves the complete Stage 1 representation. The sequence-length experiments qualify this conclusion by showing that observing more packets does not necessarily lead to better performance.

RQ2: cross-flow context. RQ2 is answered by showing that selective historical context can improve classification beyond independent flow prediction. Among the tested relations, source-host context provides the strongest overall balance, whereas endpoint context shifts the operating point towards higher recall at the cost of more false positives. Across the evaluated Stage 2 architectures, the proposed target-query model achieves the strongest overall performance.

Together, these findings answer the central research question by showing that intra-flow temporal structure and selective inter-flow context provide complementary information at different levels of the detection hierarchy. Temporal structure describes how a flow develops internally, while selective historical context relates that representation to earlier network activity. Combining these two levels improves flow-level intrusion detection on the evaluated capture. The main contribution of this thesis is therefore not the use of a Transformer architecture in isolation, but the design and evaluation of a two-stage hierarchical decomposition that explicitly combines these complementary sources of evidence.

7.2 Limitations and Scope

Evidence is limited to one organisational capture with alert-derived binary targets. Label 0 records the absence of a configured Suricata alert, not independently established benignity, and the binary target cannot reveal which attack families benefit from each relation. A fixed seed was used, leaving training variability unestimated. Small margins

between neighbouring configurations are consequently descriptive, with no claim of statistical significance [51, 52]. Transfer across networks, time periods, and rulesets has not been evaluated.

Causality in Stage 2 is defined by stable flow-start order; historical embeddings are not required to be available before the target begins. In addition, the no-context control is not capacity matched. Freezing Stage 1 clarifies experimental attribution but gives up end-to-end optimisation. Reported Stage 2 latency starts only after embedding and context-index construction, excluding PCAP processing, Suricata extraction, Stage 1 inference, production memory demand, tail latency, and concept drift.

7.3 Future Work

Two methodological directions follow directly from these limitations: resource-efficient Stage 1 pre-training and relation-aware Stage 2 context that adapts to attack behaviour. Either extension should be accompanied by stronger statistical and external validation.

7.3.1 Resource-Efficient Self-Supervised Pre-Training

Stage 1 currently learns only from supervised binary targets, restricting its representation to the signal available in alert-derived labels. Large-scale pre-training was outside the available computational budget. Future work should therefore pre-train the encoder on a broader collection of unlabelled packet and flow records before task-specific fine-tuning. Self-supervision may recover reusable traffic structure without plaintext payloads or labels for every flow [23, 4].

Several objectives fit the current feature space. Masked packet-feature modelling could reconstruct selected header, direction, length, and timing attributes. Predictive tasks could estimate a subsequent packet’s direction, discretised length, or log inter-arrival time. Temporal-order verification would distinguish authentic packet order from perturbations, while contrastive learning could align valid subsequences or augmented views of one flow. Raw addresses, ports, and similar identity fields should be removed or carefully randomised so that pre-training captures behaviour rather than host identity.

Pre-training relocates computation to an offline phase rather than removing it. Mixed precision, gradient accumulation, moderate sequence lengths, shared parameters, and progressive scaling of traffic samples should therefore be investigated. Under equal labelled-data budgets, downstream comparisons should cover random initialisation, linear probing of a frozen encoder, partial fine-tuning, and full fine-tuning. Beyond in-domain macro-F1, evaluation should ask whether pre-training improves label efficiency, rare-attack detection, convergence, and transfer to later or external captures.

7.3.2 Attack-Adaptive Relation-Aware Context Modelling

Stage 2 results demonstrate that context quality is relation dependent. Source-host history gives the best aggregate balance, whereas endpoint history exchanges additional false positives for higher attack recall. A single relation is unlikely to suit every mechanism: scanning may involve one source and many destinations, DDoS many sources and one destination, brute force repeated service interactions, and lateral movement a temporally connected endpoint chain. Context selection should therefore depend on the target and hypothesised behaviour instead of one globally fixed relation.

One extension is a causal pool containing source-host, destination-host, endpoint, protocol, service, subnet, and temporal-neighbour relations. Relation identity could enter

through learned embeddings or attention biases; alternatively, separate key–value projections could be combined by a target-conditioned gate. In both cases, eligibility should be limited to embeddings available at decision time, enforced through flow-end or embedding-availability timestamps rather than start order alone. The horizon might also adapt to traffic density by selecting between duration-based and flow-count windows.

Testing this proposal requires attack-family labels or a defensible behavioural grouping beyond the binary target. Per-family recall, FPR, confusion patterns, and relation weights would indicate whether attacks actually draw on different contexts. Appropriate controls include the current single relation, relation-agnostic attention, and parameter-matched multi-relation models. Cross-time and cross-site tests should mask host identities to prevent apparent relational gains from reflecting machines historically associated with alerts. Learned relation weights may aid diagnosis, but causal interpretation requires relation ablation and counterfactual removal of context.

7.3.3 Streaming Deployment, Efficiency, and Robustness

Although the formulation is compatible with online inference, streaming behaviour remains untested. A deployment implementation must maintain bounded buffers for each relation, evict expired entries, process out-of-order records, and define behaviour under packet loss, timestamp noise, missing features, NAT, and highly shared servers. Evaluation should span PCAP processing, Suricata extraction, both learning stages, and alert generation. Alongside predictive metrics, realistic-load tests should report throughput, peak memory, and median, 95th-percentile, and 99th-percentile latency.

Bibliography

- [1] Martin Roesch. Snort: Lightweight intrusion detection for networks. In *Proceedings of the 13th USENIX Conference on System Administration (LISA)*, 1999.
- [2] Robin Sommer and Vern Paxson. Outside the closed world: On using machine learning for network intrusion detection. In *2010 IEEE Symposium on Security and Privacy*, pages 305–316, 2010. doi:10.1109/SP.2010.25.
- [3] A. Sharma et al. A survey on encrypted network traffic: A comprehensive review. *Computer Networks*, page 110984, 2025. doi:10.1016/j.comnet.2024.110984.
- [4] Wenqi Dong, Jing Yu, Xinjie Lin, Kewei Chen, Qianwei Meng, Qingjun Yuan, and Yongjuan Wang. Deep learning and pre-training technology for encrypted traffic classification: A comprehensive review. *Neurocomputing*, 617:128444, 2025. doi:10.1016/j.neucom.2024.128444.
- [5] Open Information Security Foundation. Suricata user guide. <https://docs.suricata.io/>, 2026.
- [6] Ali Hozouri, Abbas Mirzaei, and Mehdi Effatparvar. A comprehensive survey on intrusion detection systems with advances in machine learning, deep learning and emerging cybersecurity challenges. *Discover Applied Sciences*, 2025. doi:10.1007/s44163-025-00578-1.

- [7] Anna L. Buczak and Erhan Guven. A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2):1153–1176, 2016. doi:[10.1109/COMST.2015.2494502](https://doi.org/10.1109/COMST.2015.2494502).
- [8] Mohammad Ahmed, Abdun Naser Mahmood, and Jiankun Hu. A survey of network anomaly detection techniques. *Journal of Network and Computer Applications*, 60:19–31, 2016. doi:[10.1016/j.jnca.2015.11.016](https://doi.org/10.1016/j.jnca.2015.11.016).
- [9] Chuanlong Yin, Yuefei Zhu, Jinlong Fei, and Xinzheng He. A deep learning approach for intrusion detection using recurrent neural networks. *IEEE Access*, 5:21954–21961, 2017. doi:[10.1109/ACCESS.2017.2762418](https://doi.org/10.1109/ACCESS.2017.2762418).
- [10] Nathan Shone, Tran Ngoc N. Ngoc, Vu Dinh Phai, and Qi Shi. A deep learning approach to network intrusion detection. *IEEE Transactions on Emerging Topics in Computational Intelligence*, 2(1):41–50, 2018. doi:[10.1109/TETCI.2017.2772792](https://doi.org/10.1109/TETCI.2017.2772792).
- [11] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. Kitsune: An ensemble of autoencoders for online network intrusion detection. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2018. doi:[10.14722/ndss.2018.23204](https://doi.org/10.14722/ndss.2018.23204).
- [12] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [13] Zihan Wu, Hong Zhang, Peng Wang, and Zhen Sun. Rtids: A robust transformer-based approach for intrusion detection system. *IEEE Access*, 10:64375–64387, 2022. doi:[10.1109/ACCESS.2022.3182333](https://doi.org/10.1109/ACCESS.2022.3182333).
- [14] Xueying Han, Susu Cui, Song Liu, Zhigang Lu, and Bo Jiang. Network intrusion detection based on n-gram frequency and time-aware transformer. *Computers & Security*, 127:103171, 2023. doi:[10.1016/j.cose.2023.103171](https://doi.org/10.1016/j.cose.2023.103171).
- [15] Liam Daly Manocchio, Siamak Layeghy, Wai Weng Lo, Gayan K. Kulatilleke, Mohanad Sarhan, and Marius Portmann. Flowtransformer: A transformer framework for flow-based network intrusion detection systems. *Expert Systems with Applications*, 2024. doi:[10.1016/j.eswa.2023.122564](https://doi.org/10.1016/j.eswa.2023.122564).
- [16] Shadi Aljawarneh, Muneer Aldwairi, and Muneer Yassein. Anomaly-based intrusion detection system through feature selection analysis and building hybrid efficient model. *Journal of Computational Science*, 25:152–160, 2018. doi:[10.1016/j.jocs.2017.03.006](https://doi.org/10.1016/j.jocs.2017.03.006).
- [17] Farhan Ullah, Shamsheer Ullah, Gautam Srivastava, and Jerry Chun-Wei Lin. IDS-INT: Intrusion detection system using transformer-based transfer learning for imbalanced network traffic. *Digital Communications and Networks*, 10(1):190–204, 2024. doi:[10.1016/j.dcan.2023.03.008](https://doi.org/10.1016/j.dcan.2023.03.008).
- [18] Haoyu Liu and Paul Patras. NetSentry: A deep learning approach to detecting incipient large-scale network attacks. *Computer Communications*, 191:119–132, 2022. doi:[10.1016/j.comcom.2022.04.020](https://doi.org/10.1016/j.comcom.2022.04.020).
- [19] Zhen Long et al. A transformer-based network intrusion detection approach for high-dimensional imbalanced data. *Journal of Cloud Computing*, 2024.

- [20] Xuefeng Jiang, Liuquan Xu, and Li Yu. MFT: A novel memory flow transformer efficient intrusion detection method. *Computers & Security*, 148:104174, 2025. doi:[10.1016/j.cose.2024.104174](https://doi.org/10.1016/j.cose.2024.104174).
- [21] Xinjie Lin, Gang Xiong, Gaopeng Gou, Zhen Li, Junzheng Shi, and Jing Yu. Etbert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference*, pages 633–642, 2022.
- [22] Ruijie Zhao, Mingwei Zhan, Xianwen Deng, Fangqi Li, et al. A novel self-supervised framework based on masked autoencoder for traffic classification. *IEEE/ACM Transactions on Networking*, 32(3):2012–2025, 2024. doi:[10.1109/TNET.2023.3335253](https://doi.org/10.1109/TNET.2023.3335253).
- [23] Ke Xu, Xixi Zhang, Yu Wang, Tomoaki Ohtsuki, Bamidele Adebisi, Hikmet Sari, and Guan Gui. Self-supervised learning malware traffic classification based on masked autoencoder. *IEEE Internet of Things Journal*, 11(10):17330–17340, 2024. doi:[10.1109/JIOT.2024.3357072](https://doi.org/10.1109/JIOT.2024.3357072).
- [24] Longlong Li, Yuliang Lu, Guozheng Yang, and Xuehu Yan. End-to-end network intrusion detection based on contrastive learning. *Sensors*, 24(7):2122, 2024. doi:[10.3390/s24072122](https://doi.org/10.3390/s24072122).
- [25] Hong Huang, Ye Lu, Shaohua Zhou, Xingxing Zhang, and Ze Li. CoTNeT: Contextual transformer network for encrypted traffic classification. *Egyptian Informatics Journal*, 26:100475, 2024. doi:[10.1016/j.eij.2024.100475](https://doi.org/10.1016/j.eij.2024.100475).
- [26] Qingyang Li, Yihang Zhang, Zhidong Jia, Yannan Hu, Lei Zhang, Jianrong Zhang, Yongming Xu, Yong Cui, Zongming Guo, and Xinggong Zhang. Dollm: How large language models understanding network flow data to detect carpet bombing ddos. *arXiv preprint arXiv:2405.07638*, 2024.
- [27] Xianwen Deng, Jiayi Zhu, Xiaolong Pei, Lei Zhang, Zheming Ling, and Kui Ren Xue. Flow topology-based graph convolutional network for intrusion detection in label-limited iot networks. *IEEE Transactions on Network and Service Management*, 20(1):684–696, 2023. doi:[10.1109/TNSM.2022.3213807](https://doi.org/10.1109/TNSM.2022.3213807).
- [28] Renjie Xu, Guangwei Wu, Weiping Wang, Xing Gao, An He, and Zhengpeng Zhang. Applying self-supervised learning to network intrusion detection for network flows with graph neural network. *arXiv preprint arXiv:2403.01501*, 2024.
- [29] Mei-Hui Zhong, Mingwei Lin, Chao Zhang, et al. A survey on graph neural networks for intrusion detection systems: Methods, trends and challenges. *Computers & Security*, 141:103821, 2024. doi:[10.1016/j.cose.2024.103821](https://doi.org/10.1016/j.cose.2024.103821).
- [30] Sergio Martin-Reizabal, Adrian Caballero-Quiroga, Beatriz Gil-Arroyo, Nuño Basurto, and Ruben Ruiz-Gonzalez. Transformer-based classification of IoT network traffic with flow-to-window aggregation. *Internet of Things*, 36:101879, 2026. doi:[10.1016/j.iot.2026.101879](https://doi.org/10.1016/j.iot.2026.101879).
- [31] Abdelkader El Mahdaouy, Issam Ait Yahia, Soufiane Oualil, and Ismail Berrada. Deep learning for contextualized NetFlow-based network intrusion detection: Methods, data, evaluation and deployment. *arXiv preprint arXiv:2602.05594*, 2026. doi:[10.48550/arXiv.2602.05594](https://doi.org/10.48550/arXiv.2602.05594).

- [32] Ali Shiravi, Hadi Shiravi, Mahbod Tavallaei, and Ali A. Ghorbani. Toward developing a systematic approach to generate benchmark datasets for intrusion detection. *Computers & Security*, 31(3):357–374, 2012. doi:[10.1016/j.cose.2011.12.012](https://doi.org/10.1016/j.cose.2011.12.012).
- [33] Markus Ring, Sarah Wunderlich, Deniz Scheuring, Dieter Landes, and Andreas Hotho. A survey of network-based intrusion detection data sets. *Computers & Security*, 86:147–167, 2019. doi:[10.1016/j.cose.2019.06.005](https://doi.org/10.1016/j.cose.2019.06.005).
- [34] Patrik Goldschmidt and Daniela Chudá. Network intrusion datasets: A survey, limitations, and recommendations. *Computers & Security*, 2025.
- [35] Daniela Pinto, Ivone Amorim, Eva Maia, and Isabel Praça. A review on intrusion detection datasets: Tools, processes, and features. *Computer Networks*, 262:111177, 2025. doi:[10.1016/j.comnet.2025.111177](https://doi.org/10.1016/j.comnet.2025.111177).
- [36] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, pages 108–116. SciTePress, 2018. doi:[10.5220/0006639801080116](https://doi.org/10.5220/0006639801080116).
- [37] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. Snorkel: Rapid training data creation with weak supervision. *Proceedings of the VLDB Endowment*, 11(3):269–282, 2017. doi:[10.14778/3157794.3157797](https://doi.org/10.14778/3157794.3157797).
- [38] Curtis G. Northcutt, Lu Jiang, and Isaac L. Chuang. Confident learning: Estimating uncertainty in dataset labels. *Journal of Artificial Intelligence Research*, 70:1373–1411, 2021. doi:[10.1613/JAIR.1.12125](https://doi.org/10.1613/JAIR.1.12125).
- [39] Takaya Saito and Marc Rehmsmeier. The precision–recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLOS ONE*, 10(3):e0118432, 2015. doi:[10.1371/journal.pone.0118432](https://doi.org/10.1371/journal.pone.0118432).
- [40] Daniel Arp, Erwin Quiring, Feargus Pendlebury, Alexander Warnecke, Fabio Pierazzi, Christian Wressnegger, Lorenzo Cavallaro, and Konrad Rieck. Dos and don’ts of machine learning in computer security. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 3971–3988. USENIX Association, 2022. URL: <https://www.usenix.org/conference/usenixsecurity22/presentation/arp>.
- [41] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. FiLM: Visual reasoning with a general conditioning layer. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 2018. doi:[10.1609/aaai.v32i1.11671](https://doi.org/10.1609/aaai.v32i1.11671).
- [42] Tsung-Yi Lin, Priya Goyal, Ross Girshick, Kaiming He, and Piotr Dollár. Focal loss for dense object detection. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pages 2980–2988, 2017.
- [43] Yoon Kim. Convolutional neural networks for sentence classification. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1746–1751. Association for Computational Linguistics, 2014. doi:[10.3115/v1/D14-1181](https://doi.org/10.3115/v1/D14-1181).
- [44] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. doi:[10.1162/neco.1997.9.8.1735](https://doi.org/10.1162/neco.1997.9.8.1735).

- [45] Kyunghyun Cho, Bart van Merriënboer, Çağlar Gülçehre, Dzmitry Bahdanau, Fethi Bougares, Holger Schwenk, and Yoshua Bengio. Learning phrase representations using RNN encoder–decoder for statistical machine translation. In *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 1724–1734. Association for Computational Linguistics, 2014. doi:[10.3115/v1/D14-1179](https://doi.org/10.3115/v1/D14-1179).
- [46] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *7th International Conference on Learning Representations (ICLR)*, 2019. URL: <https://openreview.net/forum?id=Bkg6RiCqY7>.
- [47] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake VanderPlas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. URL: <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [48] Tom Fawcett. An introduction to ROC analysis. *Pattern Recognition Letters*, 27(8):861–874, 2006. doi:[10.1016/j.patrec.2005.10.010](https://doi.org/10.1016/j.patrec.2005.10.010).
- [49] Google. Google colab: Frequently asked questions. Google Research, 2026. Accessed 14 July 2026. URL: <https://research.google.com/colaboratory/faq.html>.
- [50] NVIDIA Corporation. NVIDIA L4 GPU Accelerator product brief. Technical Report PB-11316-001-v01, NVIDIA Corporation, 2023. URL: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/l4/PB-11316-001_v01.pdf.
- [51] Nils Reimers and Iryna Gurevych. Reporting score distributions makes a difference: Performance study of LSTM-networks for sequence tagging. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing*, pages 338–348, Copenhagen, Denmark, 2017. Association for Computational Linguistics. URL: <https://aclanthology.org/D17-1035/>, doi:[10.18653/v1/D17-1035](https://doi.org/10.18653/v1/D17-1035).
- [52] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, Assya Trofimov, Brennan Nichyporuk, Justin Szeto, Nazanin Mohammadi Sepahvand, Edward Raff, Kanika Madan, Vikram Voleti, Samira Ebrahimi Kahou, Vincent Michalski, Tal Arbel, Chris Pal, Gaël Varoquaux, and Pascal Vincent. Accounting for variance in machine learning benchmarks. *Proceedings of Machine Learning and Systems*, 3, 2021. URL: <https://proceedings.mlsys.org/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html>.